

CANON OPERATIVO, CONTRACTUAL, DE CERTIFICACIÓN Y RESISTENCIA A LA FALSEDAD DE BAGO

Versión 1.0-RC4 consolidada y sustituible

Estado del documento: candidato consolidado a versión estable.

Sustituye íntegramente:

versión 1.0-RC1;

versión 1.0-RC2;

versión 1.0-RC3;

enmiendas anteriores sobre rutas, contratos, bienvenida, menú, centros operativos, limpieza progresiva y resistencia a la falsedad.

Objetivo: definir una base operacional fiable, trazable, verificable, reproducible, certificable y estructuralmente resistente a la creación, propagación y ocultación de estados falsos.

1. PROPÓSITO

El objetivo de BAGO no es aparentar capacidad, sino demostrarla.

BAGO deberá poder identificar de forma verificable:

dónde está operando;

qué instalación del framework utiliza;

qué proyecto está manejando;

qué workspace está vinculado;

qué alcance de archivos tiene autorizado;

qué sesión está activa;

qué objetivo mantiene;

qué decisiones siguen vigentes;

qué restricciones debe respetar;

qué provider, adapter, runtime y modelo ejecutan cada petición;

qué contexto se prepara;

qué contexto se entrega realmente al modelo;

qué información se recupera;

qué archivos se utilizan como evidencia;

qué herramientas están disponibles;

qué herramientas están autorizadas;

qué acciones se solicitan;

qué acciones se ejecutan;

qué resultados se validan;

qué límites se han medido;

qué capacidades están implementadas;

qué capacidades están verificadas;

qué capacidades están certificadas;

qué capacidades continúan pendientes.

BAGO no solo deberá demostrar la verdad después de afirmar algo.

Deberá dificultar estructuralmente:

la creación de estados falsos;

la simulación de ejecuciones;

la presentación de datos no confirmados como efectivos;

la fabricación de receipts;

la alteración de evidencias;

la ocultación de contradicciones;

la autocertificación;

la propagación de falsedades entre capas.

La prioridad será la certeza por encima de la velocidad.

Ningún componente podrá declarar como verdadero un estado, una capacidad, una ejecución o un resultado que no esté respaldado por evidencia reproducible.

2. RESULTADO ESPERADO DE ESTA ITERACIÓN

Esta iteración deberá producir una base operacional fiable.

BAGO deberá:

- conocer la instalación activa del framework;
- conocer el proyecto vinculado;
- conocer el workspace operacional;
- distinguir framework, proyecto, workspace y alcance autorizado;
- mantener sesión, objetivo, decisiones y restricciones;
- resolver el estado real antes de presentar acciones;
- construir contexto explícito antes de cada llamada relevante al modelo;
- recuperar información real del proyecto cuando sea necesaria;
- emitir receipts verificables;
- representar únicamente estados confirmados por el backend;
- utilizar contratos compartidos entre Terminal, API y React;
- mantener un único registro canónico de acciones;
- presentar un menú compacto mediante centros operativos;
- mantener comandos directos para automatización y acceso rápido;
- medir inicialmente sus límites reales;
- detectar comportamientos falsos, simulados o incoherentes;
- evitar afirmaciones no respaldadas;

dificultar la construcción de estados inválidos;

detectar alteraciones de evidencias y eventos;

contener contradicciones;

atribuir inconsistencias a su componente de origen;

permitir verificación adversarial;

preservar las funciones existentes;

adaptar la bienvenida y el menú al estado real;

mantener separadas presentación, transporte, lógica y autoridad.

El cierre de esta iteración no significará que todo BAGO esté terminado.

Significará que BAGO dispone de una base fiable sobre la que se pueden construir capacidades posteriores sin confundir:

apariencia;

ejecución;

verificación;

certificación;

confianza;

verdad.

3. NO OBJETIVOS DE ESTA ITERACIÓN

No serán condiciones obligatorias para cerrar esta iteración:

certificar todos los modelos;

activar todos los providers cloud;

completar toda la autoevolución;

implementar autonomía total;

certificar todas las configuraciones de hardware;

certificar todos los tamaños posibles de contexto;

rediseñar completamente la interfaz;

reescribir BAGO desde cero;

sustituir componentes funcionales por arquitecturas paralelas;

implementar aprendizaje automático complejo cuando una solución verificable más simple sea suficiente;

hacer que React ejecute necesariamente el REPL real;

mostrar todos los comandos registrados en el menú visible;

crear una rama independiente por cada verbo operativo;

utilizar blockchain;

implementar criptografía distribuida cuando una cadena local de integridad sea suficiente;

cerrar el canon completo.

Solo serán obligatorios los criterios mínimos definidos para la base operacional.

4. DEFINICIONES CANÓNICAS

4.1. Provider

Provider es la fuente lógica o servicio de inferencia registrado en BAGO.

Ejemplos conceptuales:

ollama-local;

ollama-cloud;

anthropic;

openrouter.

4.2. Adapter

Adapter es el componente de BAGO que integra un provider o runtime con los contratos internos del framework.

Traduce:

mensajes;

parámetros;

herramientas;

errores;

streaming;

respuestas;

metadatos.

4.3. Runtime

Runtime es el motor que carga y ejecuta realmente un modelo.

Ejemplos:

Ollama;

llama.cpp;

otro motor local o remoto.

4.4. Modelo

Modelo es la implementación concreta utilizada para una petición.

Ejemplo conceptual:

llama3.2:3b.

4.5. Configuración efectiva

Configuración efectiva es el conjunto real de parámetros aplicados durante una ejecución.

Puede incluir:

contexto;

cuantización;

batch;

paralelismo;

capas descargadas en GPU;

tipo de caché KV;

tokens máximos de salida;

temperatura;

plantilla de conversación;

prompt del sistema;

herramientas;

políticas operativas.

4.6. Framework root

Framework root es la instalación canónica de BAGO.

Su raíz canónica será la carpeta `.bago` de la instalación activa o del repositorio principal de BAGO.

Ejemplo conceptual:

C:\...\BAG4.8.bago

Framework root contendrá componentes genéricos y reutilizables:

runtime interno;

comandos;

dispatcher;

adaptadores;

contratos;

gestión de sesiones;

gestión de contexto;

bridge;

telemetría;

logging;

herramientas genéricas;

agentes genéricos;

benchmarks comunes;

servicios de certificación;

migraciones;

esquemas.

Framework root no contendrá estado específico de un proyecto externo.

4.7. Project root

Project root es la raíz real del proyecto o repositorio vinculado.

Contiene normalmente:

código fuente;

documentación;

recursos;

configuración propia;

tests;

archivos funcionales del proyecto.

Project root no es framework root.

Project root tampoco es workspace root.

4.8. Workspace

Workspace es la unidad operacional vinculada a un proyecto.

Su estado canónico vive en:

`workspace_root = project_root/.gabo`

El workspace contiene:

identidad del proyecto;

manifiesto;

estado operacional;

memoria;

índice;

recuperación;

decisiones;

restricciones;

resúmenes;

receipts;

certificaciones;

artefactos específicos del proyecto.

Workspace no contiene código del framework salvo utilidades explícitas de:

migración;

compatibilidad;

siembra;

diagnóstico temporal.

Estas utilidades deberán estar marcadas y no podrán convertirse en autoridad del framework.

4.9. Workspace root

Workspace root es la carpeta `.gabo` del proyecto vinculado.

Su forma canónica será:

`workspace_root = project_root/.gabo`

En `.gabo` vivirán los datos y artefactos específicos del proyecto.

No deberá utilizarse `.gabo` para almacenar componentes genéricos que pertenecen al framework.

4.10. Workspace scope root

Workspace scope root es el límite autorizado para que BAGO pueda:

recuperar;

leer;

modificar;

ejecutar;

utilizar archivos como evidencia.

Normalmente:

`workspace_scope_root = project_root`

Puede definirse un alcance más limitado cuando la política del workspace lo requiera.

Workspace scope root no deberá ampliarse silenciosamente.

4.11. Sesión

Sesión es la unidad persistente de conversación y operación.

Debe conservar:

`session_id`;

`workspace_id`;

`framework_root`;

`project_root`;

`workspace_root`;

`workspace_scope_root`;

`objetivo`;

`decisiones`;

`restricciones`;

`provider`;

adapter;

runtime;

modelo;

modo;

herramientas autorizadas;

context_revision.

4.12. Evidencia

Evidencia es un dato observable que permite verificar una afirmación.

Puede proceder de:

efecto real;

respuesta estructurada;

persistencia;

logs;

runtime;

medición;

test;

archivo;

base de datos;

receipt.

4.13. Receipt

Receipt es un registro estructurado de una operación.

No es una explicación narrativa.

Debe contener:

identidad;

entrada;

estado;

resultado;

evidencias;

advertencias;

huella;

identificador de ejecución.

4.14. Claim

Claim es una afirmación concreta y verificable sobre el comportamiento o estado de BAGO.

Cada claim deberá disponer de:

identificador;

criterio de aceptación;

test;

resultado;

evidencia;

estado de madurez;

estado de certificación.

4.15. Manifiesto del workspace

El manifiesto será la fuente canónica de identidad del workspace dentro de `.gabo`.

Su ubicación canónica será:

`project_root/.gabo/workspace.json`

Deberá contener como mínimo:

`workspace_id`;

`project_root`;

workspace_root;

workspace_scope_root;

schema_version;

fecha de creación;

estado de migración;

revisión;

política de acceso.

4.16. Contrato de frontera

Contrato de frontera es el esquema versionado que define los datos intercambiados entre:

core;

API;

Terminal;

React;

herramientas;

presentadores.

No será una descripción informal.

Deberá poder validarse automáticamente.

4.17. Presenter

Presenter es la capa que transforma el estado canónico del backend en una representación para una superficie concreta.

BAGO podrá tener:

TerminalPresenter;

ReactPresenter;

otros presentadores futuros.

Un presenter no será autoridad operacional.

4.18. DTO canónico

DTO canónico es una representación estructurada y versionada de un estado o resultado.

Los DTO canónicos serán producidos por el backend y validados por sus consumidores.

4.19. Acción permitida

Acción permitida es una operación autorizada por el backend para el estado actual.

La UI no decidirá por sí sola si una acción es válida.

4.20. Canal de origen

Canal de origen identifica desde qué superficie llegó una solicitud.

Ejemplos:

terminal_repl;

react_ui;

api_client.

El canal podrá utilizarse para:

telemetría;

presentación;

diagnóstico;

preferencias de salida.

No podrá modificar por sí mismo:

permisos;

workspace;

modelo efectivo;

autoridad;

certificación.

4.21. Registro canónico de comandos

Registro canónico de comandos es la autoridad de toda capacidad ejecutable expuesta por BAGO.

Define:

command_id;

nombre;

argumentos;

permisos;

efectos;

estados permitidos;

confirmaciones;

servicio responsable;

receipt esperado.

4.22. Centro operativo

Centro operativo es la superficie compacta que reúne, para un dominio:

estado;

colección;

selección;

acciones permitidas;

acciones bloqueadas;

advertencias;

actividad reciente;

detalle contextual.

No es una carpeta vacía ni una lista mecánica de verbos.

4.23. Árbol visible

Árbol visible es la proyección contextual del registro canónico que TerminalPresenter o ReactPresenter muestran al usuario.

El árbol visible no equivale al registro canónico.

4.24. Comando directo

Comando directo es una invocación reproducible de una acción canónica.

Se utiliza para:

acceso rápido;

automatización;

scripts;

tests;

documentación.

Debe converger en la misma implementación que la acción interactiva equivalente.

4.25. Falsedad operacional

Falsedad operacional es cualquier representación que induzca a interpretar como real, ejecutado, confirmado, verificado o certificado algo que no ha sido demostrado.

Puede ser:

intencional;

accidental;

producto de un bug;

producto de un contrato ambiguo;

producto de datos obsoletos;

producto de una interfaz optimista;

producto de una excepción silenciada;

producto de una estimación mal clasificada.

No es necesario demostrar intención de engaño.

4.26. Testigo operacional

Testigo operacional es un componente que observa una ejecución sin ser su ejecutor principal.

Puede verificar:

efectos;

archivos;

procesos;

telemetría;

persistencia;

receipts;

resultados.

4.27. ContradictionRecord

ContradictionRecord es el registro canónico de una contradicción entre dos fuentes o estados.

4.28. Capability

Capability es una autorización explícita, limitada, verificable y revocable para realizar una clase concreta de efectos.

4.29. Cadena de integridad

Cadena de integridad es una secuencia de eventos donde cada evento referencia mediante hash al anterior.

Permite detectar:

eliminación;

alteración;

inserción;

reordenación.

4.30. Estado epistemológico

Estado epistemológico clasifica la calidad y origen de un dato.

Estados:

observed;

verified;

calculated;

estimated;

inferred;

proposed;

unknown;

not_available;

contradicted.

5. AUTORIDAD DE RUTAS Y CAPAS

BAGO deberá separar estrictamente:

framework;

proyecto;

workspace;

alcance autorizado;

sesión;

ejecución.

5.1. Política canónica

`.bago` representa el framework BAGO.

`project_root` representa la raíz real del proyecto.

`.gabo` representa el estado operacional del workspace.

`workspace_scope_root` representa el alcance autorizado de operaciones.

Los componentes genéricos para todos los workspaces vivirán en framework root.

Los artefactos específicos del proyecto vivirán en workspace root.

El código y los archivos normales del proyecto vivirán en project root.

5.2. Componentes del framework

En `.bago` vivirán componentes reutilizables:

comandos;

adaptadores;

contratos;

servicios;

runtime interno;

herramientas genéricas;

telemetría;

certificación;

gestión de contexto;

gestión de sesiones.

Ningún artefacto específico de un proyecto podrá utilizar framework root como almacenamiento canónico.

5.3. Componentes del proyecto

En project root vivirán:

código;

documentación;

recursos;

tests;

configuración propia.

Un archivo del proyecto podrá utilizarse como evidencia cuando:

pertenezca al workspace vinculado;

esté dentro de workspace_scope_root;

sea accesible;

su revisión esté vigente.

5.4. Componentes del workspace

En `.gabo` vivirán:

manifiesto;

workspace_id;

memoria;

índice;

recuperación;

estado operacional;

receipts;

resúmenes;

decisiones;

certificaciones específicas;

artefactos generados para ese proyecto.

La presencia de una herramienta dentro de `.gabo` no la convertirá en herramienta global del framework.

5.5. Regla de ubicación

Si una pieza vive en `.bago` y es específica de un proyecto, estará mal ubicada.

Si una pieza vive en `.gabo` y es genérica para todos los workspaces, estará mal ubicada.

Si un archivo está fuera de `workspace_scope_root`, no podrá utilizarse como evidencia sin autorización explícita.

5.6. Prohibiciones

La UI no podrá convertir una ruta local en autoridad por el hecho de mostrarla.

La Terminal no podrá heredar automáticamente un proyecto desde el directorio actual.

La API no podrá aceptar una ruta del cliente como fuente de verdad sin validarla contra la sesión y el manifiesto.

Un workspace no podrá redefinir framework root.

Framework root no podrá contener estado canónico de un proyecto externo.

6. COMPATIBILIDAD LEGACY

Una carpeta `.bago` situada dentro de un proyecto externo no será el formato canónico del workspace.

Solo podrá reconocerse para:

detección legacy;

lectura durante migración;

conversión a `.gabo` ;

compatibilidad temporal;

recuperación controlada de datos antiguos.

Una `.bago` legacy dentro de un proyecto:

no será fuente primaria de estado;

no competirá con `.gabo` ;

no podrá convertirse en framework root;

no recibirá nuevos artefactos canónicos;

no sobrescribirá el manifiesto de `.gabo` .

Si `.bago` y `.gabo` existen simultáneamente:

`.gabo` será la autoridad;

`.bago` será considerada legacy;

las discrepancias se registrarán;

no se fusionarán estados silenciosamente.

Después de migrar, la carpeta legacy deberá quedar:

archivada;

marcada como migrada;

ignorada;

o eliminada mediante autorización explícita.

7. PRINCIPIO FUNDAMENTAL

Toda afirmación operativa deberá seguir esta cadena:

afirmación;

condición verificable;

ejecución;

evidencia;

validación;

conclusión;

receipt.

La fórmula fundamental será:

AFIRMACIÓN VÁLIDA = CONDICIÓN VERIFICABLE MÁS EVIDENCIA OBSERVABLE MÁS VALIDACIÓN
REPRODUCIBLE

La existencia de código no demuestra funcionamiento.

La existencia de una interfaz no demuestra ejecución del backend.

La ausencia de excepciones no demuestra corrección.

Una respuesta HTTP satisfactoria no demuestra que la operación interna sea válida.

Un test positivo aislado no demuestra resistencia a fallos.

Una captura de pantalla no demuestra el estado real del backend.

Una explicación del modelo no es evidencia suficiente.

8. PRINCIPIO TRANSVERSAL DE RESISTENCIA A LA FALSEDAD

BAGO no confiará únicamente en que sus componentes declaren la verdad.

La arquitectura deberá dificultar:

construir estados falsos;

persistirlos;

propagarlos;

mostrarlos como confirmados;

ocultar sus contradicciones;

eliminar sus rastros.

La resistencia a la falsedad tendrá cuatro niveles:

prevención;

detección;

contención;

atribución.

La fórmula será:

RESISTENCIA A LA FALSEDAD = PREVENCIÓN MÁS DETECCIÓN MÁS CONTENCIÓN MÁS ATRIBUCIÓN

8.1. Prevención

BAGO deberá utilizar:

tipos cerrados;

constructores controlados;

esquemas;

precondiciones;

capabilities;

transiciones válidas;

mínimos privilegios.

8.2. Detección

BAGO deberá utilizar:

validadores;

testigos;

comparación de fuentes;

hashes;

mutaciones;

pruebas adversariales;

motor de coherencia.

8.3. Contención

Una falsedad detectada no deberá propagarse a:

sesión;

contexto;

pipeline;

certificación;

herramientas;

interfaz.

8.4. Atribución

Toda inconsistencia deberá poder vincularse a:

component_id;

request_id;

execution_id;

session_id;

workspace_id;

actor;

contrato;

evento.

9. PRINCIPIOS OPERATIVOS OBLIGATORIOS

BAGO deberá:

medir antes de optimizar;

diferenciar valores declarados, configurados, aplicados, observados, calculados, estimados y certificados;

mantener una única autoridad de estado;

no inventar rutas, archivos, módulos, herramientas, resultados ni capacidades;

distinguir evidencia, inferencia, propuesta, intención y ejecución;

mantener trazabilidad entre afirmación, fuente, evidencia, cálculo y conclusión;

no modificar configuraciones silenciosamente;

reutilizar la arquitectura existente antes de crear sistemas nuevos;

no certificar con una sola ejecución;

no permitir que una media oculte fallos críticos;

no truncar silenciosamente instrucciones, decisiones o restricciones;

mantener cada sesión vinculada a un workspace real;

validar las autoridades de ruta antes de operar;

fallar de forma segura;

declarar expresamente lo que no conoce;

permitir reversión de cambios importantes;

preservar las capacidades existentes por defecto;

mostrar únicamente acciones coherentes con el estado real;

mantener React y Terminal como presentadores del mismo estado canónico;

no duplicar contratos, menús ni registros de comandos;

validar payloads en productores y consumidores;

separar el registro ejecutable del árbol visible;

unificar estado, listado, inspección y selección cuando la carga lo permita;

crear subniveles solo por complejidad demostrada;

construir estados críticos únicamente mediante servicios canónicos;

registrar contradicciones;

conservar fallos históricos;

evitar excepciones silenciosas;

aplicar privilegio mínimo;

verificar efectos después de ejecutar;

distinguir receipt emitido de receipt validado.

10. SEPARACIÓN DE ESTADOS

Nunca se utilizará un único estado para representar ejecución, madurez, certificación, presentación, selección y calidad epistemológica.

10.1. Estado de ejecución

Estados:

pending;

validated;

authorized;

running;

done;

failed;

blocked;

cancelled.

Done significará:

ejecución terminada;

salida validada;

evidencia presente;

receipt validado;

ausencia de error bloqueante.

Done no significará certified.

10.2. Estado de madurez

Estados:

not_implemented;

implemented_unverified;

partially_tested;

verified_in_isolation;

verified_in_integration.

10.3. Estado de certificación

Estados:

not_requested;

pending;

certified;

rejected;

invalidated.

Certified significará:

verified_in_integration;

separación de roles;

pruebas adversariales;

repeticiones suficientes;

resultado reproducible;

receipt completo;

fingerprint válida;

ausencia de bloqueos críticos.

10.4. Estado de presentación

La UI podrá utilizar:

unknown;

loading;

confirmed;

degraded;

error;

stale;

contradicted.

10.5. Estado de selección

La selección podrá estar en:

visual;

pending_confirmation;

effective;

rejected;

stale.

Mover el foco visual no producirá un cambio operacional.

10.6. Estado epistemológico

Los datos deberán poder clasificarse como:

observed;

verified;

calculated;

estimated;

inferred;

proposed;

unknown;

not_available;

contradicted.

Una inferencia no se mostrará como observación.

Una propuesta no se mostrará como ejecución.

11. ESTADOS IMPOSIBLES DE REPRESENTAR

BAGO deberá diseñar los contratos para impedir o rechazar representaciones inválidas.

No será válido:

PipelineState done sin execution_id, receipt y evidence_ids;

ClaimState verified sin tests ni evidencias;

CertificationState certified sin fingerprint;

WorkspaceState linked_confirmed sin manifiesto válido;

modelo efectivo sin provider, adapter y runtime efectivos;

ToolState completed sin ToolReceipt;

ContextState confirmed sin ContextEnvelope válido;

ContextReceipt con identidad diferente del envelope;

selección efectiva sin confirmación del backend;

FileMutationState applied sin hash posterior validado.

La validación deberá fallar antes de:

persistir;

propagar;

mostrar;

certificar.

12. PROHIBICIÓN DE AUTOCERTIFICACIÓN

El implementador no podrá declarar certificada su propia implementación basándose únicamente en su conclusión.

Podrá declarar:

implementado sin verificar;

probado parcialmente;

verificado en aislamiento;

verificado en integración;

pendiente de certificación.

La certificación exigirá separación efectiva mediante:

otro modelo;

otro agente;

otra sesión limpia;

otro proceso;

un harness determinista;

una suite adversarial protegida;

una revisión humana reproducible.

No será obligatorio utilizar otro modelo.

Sí será obligatorio que el certificador pueda intentar falsar la afirmación.

Un validador interno podrá declarar:

valid;

invalid;

complete;

incomplete.

No podrá convertir automáticamente esos estados en certified.

13. PROHIBICIÓN DE AUTOAFIRMACIÓN

Un componente no podrá utilizar su propia afirmación como evidencia suficiente.

No serán evidencias suficientes:

el modelo afirma haber utilizado una herramienta;

la UI afirma que el pipeline terminó;

el adapter afirma que usó el modelo correcto;

el comando afirma que escribió un archivo;

el certificador interno afirma que todo está certificado.

La fórmula será:

AUTOINFORME NO EQUIVALE A EVIDENCIA

La evidencia deberá proceder de:

efecto observado;

telemetría;

persistencia;

testigo operacional;

hash;

runtime;

servicio independiente.

14. REGISTRO CANÓNICO DE CLAIMS

Cada afirmación verificable tendrá un claim_id único.

El registro deberá contener:

claim_id;

capacidad;

afirmación;

sujeto;

alcance;

críticidad;

componente responsable;

creado por;

fecha de creación;

condición de aceptación;

evidencias requeridas;

tests asociados;

comandos;

entradas;

resultado esperado;

resultado observado;

código de salida;

evidence_ids;

contradiction_ids;

CertificationFingerprint;

estado de ejecución;

estado de madurez;

estado de certificación;

fecha;

condiciones de caducidad;

causa de invalidación.

Una afirmación sin prueba no podrá considerarse aceptada.

Una afirmación sin evidencia no podrá mostrarse como confirmada.

La UI no podrá crear claims canónicos aprobados.

Podrá crear:

borradores;

observaciones;

solicitudes de validación.

El backend será responsable de registrar el claim canónico.

15. CRITICIDAD

15.1. Critical

Puede:

dañar el workspace;

violar permisos;

falsear evidencias;

usar el modelo equivocado;

ejecutar acciones no autorizadas;

inventar resultados;

certificar incorrectamente;

perder restricciones de seguridad;

ocultar una contradicción;

alterar una cadena de integridad.

Bloquea certificación y cierre de iteración.

15.2. High

Rompe el flujo principal o produce un estado operacional incorrecto.

Bloquea la capacidad afectada.

15.3. Medium

Degrada precisión, rendimiento o experiencia sin perder seguridad o trazabilidad.

Puede quedar pendiente si se documenta.

15.4. Low

Problema secundario o visual sin impacto operacional relevante.

Puede quedar como deuda declarada.

16. FUENTE ÚNICA DE VERDAD

El backend será la autoridad del estado operacional.

Los datos canónicos serán:

framework_version;

framework_root;

project_root;

workspace_id;

workspace_root;

workspace_scope_root;

workspace_state;

session_id;

provider;

adapter;

runtime;

model;

operating_mode;

index_status;

context_revision;

configured_context;

occupied_context;

available_context;

pipeline_status;

contract_version;

allowed_actions;

effective_selection;

contradictions;

evidence_state.

16.1. Framework root

Solo podrá proceder de la instalación activa de BAGO y del backend.

La UI, el workspace o el directorio actual no podrán redefinirlo.

16.2. Project root

Deberá proceder del manifiesto del workspace y de la sesión vinculada.

El backend deberá comprobar que:

existe;

es accesible;

corresponde al workspace_id;

se relaciona con workspace_root;

está dentro de la política autorizada.

16.3. Workspace root

Deberá proceder de:

la sesión;

el manifiesto de `.gabo`;

la validación del backend.

No podrá derivarse únicamente de una selección visual.

16.4. Workspace scope root

Deberá proceder del manifiesto y de la política del workspace.

No podrá ampliarse sin autorización.

16.5. Veredicto único de binding

Deberá existir un único resolver canónico de binding.

Ese resolver producirá el WorkspaceState utilizado por:

estado de sesión;

ContextEnvelope;

ContextReceipt;

API;

Terminal;

React;

pipeline;

herramientas;

recuperación.

Ningún componente podrá volver a calcular por separado si el workspace está confirmado.

16.6. Autoridad no equivale a infalibilidad

El backend será autoridad operacional, pero no se asumirá que siempre es correcto.

Sus afirmaciones críticas deberán poder contrastarse mediante:

efectos observados;

testigos;

persistencia;

runtime;

hashes;

pruebas adversariales.

16.7. Regla de incoherencia

Cuando exista una contradicción, no se elegirá silenciosamente el valor más conveniente.

Deberá crearse un ContradictionRecord.

Cuando el backend no pueda demostrar un valor, deberá declarar:

unknown;

invalid;

blocked;

not_available;

contradicted.

17. WORKSPACESTATE CANÓNICO

El backend deberá producir un único WorkspaceState estructurado.

Los estados canónicos serán:

linked_confirmed;

detected_unlinked;

invalid;

absent;

legacy_only;

blocked.

WorkspaceState deberá contener:

state;

framework_root;

project_root;

workspace_root;

workspace_scope_root;

workspace_id;

session_id;

manifest_status;

binding_confirmed;

binding_reason;

allowed_actions;

blocked_operations;

context_revision;

warnings;

contradiction_ids;

contract_version.

17.1. Linked confirmed

Existe `.gabo`.

El manifiesto es válido.

La sesión está vinculada.

Las raíces son coherentes.

Las operaciones autorizadas pueden continuar.

17.2. Detected unlinked

Existe una `.gabo` válida.

La sesión no está vinculada a ese workspace.

Debe ofrecerse vinculación.

No debe ofrecerse inicialización.

17.3. Invalid

Existe `.gabo`, pero el manifiesto, las raíces o la revisión son incoherentes.

Las operaciones dependientes del proyecto deberán bloquearse.

17.4. Absent

No existe `.gabo`.

Puede ofrecerse inicialización.

17.5. Legacy only

Existe una `.bago` legacy dentro del proyecto y no existe `.gabo`.

Puede ofrecerse inspección o migración explícita.

17.6. Blocked

No puede determinarse una autoridad segura.

Las operaciones dependientes del proyecto deberán permanecer bloqueadas.

17.7. Acciones permitidas

`allowed_actions` será generado por el backend.

Terminal y React deberán representar exclusivamente las acciones permitidas.

No podrán construir una política paralela.

18. SESIÓN OPERATIVA

Toda sesión deberá estar vinculada explícitamente a:

`session_id`;

`framework_root`;

`workspace_id`;

`workspace_root`;

`project_root`;

`workspace_scope_root`.

También deberá conservar:

`repositorio`;

rama;

objetivo;

decisiones;

restricciones;

context_revision;

provider;

adapter;

runtime;

modelo;

modo;

herramientas autorizadas;

estado del índice;

último event_hash;

contradiction_ids.

Si no existe workspace_root válido, BAGO no asumirá ninguno.

Si no existe project_root válido, BAGO no operará sobre archivos del proyecto.

Si el usuario cambia de proyecto, la sesión deberá:

suspender o cerrar el vínculo actual;

seleccionar el nuevo proyecto;

localizar o crear `.gabo`;

validar el manifiesto;

registrar workspace_id;

actualizar raíces;

incrementar context_revision;

invalidar contexto y certificaciones incompatibles.

No se heredar  un proyecto por abrir BAGO.

El directorio actual solo podr  ser candidato de detecci n.

Nunca ser  autoridad final sin validaci n.

19. DETECCIÓN Y VINCULACI N DEL WORKSPACE

La detecci n deber  ser consciente del estado real.

BAGO determinar :

si framework_root es v lido;

si existe un project_root candidato;

si existe project_root/.gabo;

si existe workspace.json;

si el manifiesto es v lido;

si la sesi n est  vinculada;

si existe una `.bago` legacy;

si existen incoherencias entre rutas, manifiesto y sesi n.

La detecci n no significar  autom ticamente vinculaci n.

La vinculaci n deber  quedar registrada.

19.1. `.gabo` v lida y sesi n vinculada

BAGO deber :

mostrar el workspace actual;

mostrar el proyecto;

mostrar la sesi n;

mostrar el contexto;

permitir continuar, inspeccionar, verificar o cambiar de proyecto;

no ofrecer inicializar `.gabo` ;

no crear otra estructura.

19.2. `.gabo` válida y sesión no vinculada

BAGO deberá:

informar qué workspace ha detectado;

ofrecer vincular la sesión;

permitir ver detalles;

permitir recuperar una sesión anterior;

permitir seleccionar otro proyecto;

no ofrecer inicialización.

19.3. `.gabo` existente pero inválida

BAGO deberá:

mostrar el problema;

bloquear operaciones de proyecto;

ofrecer verificar;

ofrecer reparar;

ofrecer migrar cuando corresponda;

ofrecer modo seguro o solo lectura;

no crear silenciosamente otra `.gabo` .

19.4. `.gabo` inexistente

Solo entonces BAGO podrá ofrecer:

inicializar `.gabo` ;

vincular otro proyecto;

abrir una sesión anterior;

examinar en modo de solo lectura;

ejecutar diagnóstico.

19.5. `.bago` legacy sin `.gabo`

BAGO deberá:

informar que ha detectado un formato legacy;

ofrecer inspección;

ofrecer migración explícita;

ofrecer crear `.gabo` sin importar datos;

ofrecer modo de solo lectura;

no tratar `.bago` como workspace canónico.

20. PREFLIGHT OPERACIONAL

Antes de ejecutar una operación vinculada a un proyecto, BAGO deberá realizar un preflight canónico.

El preflight comprobará:

WorkspaceState;

SessionStatus;

session_id;

workspace_id;

framework_root;

project_root;

workspace_root;

workspace_scope_root;

manifest_status;

context_revision;

provider;

adapter;

runtime;

modelo;

permisos;

capabilities;

contratos;

herramientas autorizadas;

idempotency_key;

contradiction_ids abiertos;

invariantes del dominio.

El preflight será obligatorio antes de:

llamar al modelo con contexto del proyecto;

recuperar archivos;

ejecutar herramientas;

modificar archivos;

iniciar trabajos;

marcar pasos del pipeline como running;

emitir receipts operativos.

Si el preflight falla:

no se llamará al adapter;

no se ejecutará la herramienta;

no se modificará el proyecto;

el estado será blocked;

se devolverá un `ErrorState`;

se emitirá un evento `blocked`;

se ofrecerán únicamente acciones de recuperación autorizadas.

20.1. Conversación sin workspace

BAGO podrá permitir una conversación general sin workspace únicamente en un modo explícito no vinculado.

En ese modo:

no se presentará como conversación del proyecto;

no se utilizarán archivos del proyecto;

no se habilitarán herramientas de proyecto;

el `ContextEnvelope` declarará `workspace_state` como `unbound`;

la interfaz mostrará claramente que no existe contexto de proyecto.

Una sesión declarada como vinculada no podrá degradarse silenciosamente a chat general.

21. CONSTRUCCIÓN DEL CONTEXTO

El backend resolverá y validará el workspace activo antes de construir el prompt.

La UI podrá comunicar:

archivo seleccionado;

archivos abiertos;

vista activa;

acción solicitada.

La UI no podrá decidir por sí sola el workspace real.

Antes de llamar al modelo, el backend comprobará:

preflight válido;

`framework_root` válido;

project_root válido;

workspace_root válido;

workspace_scope_root válido;

workspace_id coherente;

sesión vinculada;

revisión vigente;

provider efectivo;

adapter efectivo;

runtime efectivo;

modelo efectivo;

permisos;

ausencia de contradicción crítica.

Si workspace_id, workspace_root y manifiesto no coinciden, el envío se bloqueará.

Si project_root y workspace_scope_root son incoherentes, las operaciones sobre archivos se bloquearán.

22. NIVELES DEL CONTEXTO

22.1. Contexto estable

Incluye:

identidad;

framework;

workspace;

objetivo;

restricciones;

decisiones;

permisos;

provider;

modelo.

22.2. Contexto dinámico

Incluye:

archivo seleccionado;

archivos abiertos;

rama;

vista activa;

tarea;

procesos;

trabajos;

errores recientes.

22.3. Contexto recuperado

Incluye:

fragmentos;

definiciones;

resultados;

evidencias;

decisiones históricas;

información localizada en el proyecto.

Solo se enviará el contenido necesario.

23. CONTEXTEVELOPE

Toda llamada relevante al modelo deberá derivarse de un ContextEnvelope verificable.

El ContextEnvelope incluirá:

contract_version;

source_of_truth_version;

session_id;

framework_version;

framework_root;

project_root;

workspace_id;

workspace_root;

workspace_scope_root;

workspace_state;

context_revision;

objetivo;

decisiones;

restricciones;

provider;

adapter;

runtime;

modelo;

modo;

archivos representados;

fragmentos recuperados;

resumen operativo;

herramientas disponibles;

herramientas autorizadas;

presupuesto de tokens;

advertencias;

contradiction_ids;

request_id;

execution_id;

event_chain_head.

Antes de enviarlo, BAGO validará:

que framework_root corresponde a la instalación activa;

que workspace_root corresponde al workspace_id;

que workspace_root equivale a project_root/.gabo;

que project_root corresponde al manifiesto;

que las rutas están dentro de workspace_scope_root;

que no se mezclan archivos del framework y del proyecto;

que context_revision sigue vigente;

que el contrato es compatible;

que no existen contradicciones críticas abiertas.

Una incoherencia producirá:

blocked;

causa estructurada;

evidencia;

ausencia de llamada al modelo.

24. CONTEXTRECEIPT

Después de la llamada al modelo deberá generarse un ContextReceipt.

Indicará:

contract_version;

source_of_truth_version;

session_id;

framework_root;

project_root;

workspace_id;

workspace_root;

workspace_scope_root;

workspace_state;

context_revision;

provider efectivo;

adapter efectivo;

runtime efectivo;

modelo efectivo;

tokens enviados;

tokens reservados;

tokens disponibles;

archivos representados;

fragmentos recuperados;

resumen cargado;

herramientas disponibles;

herramientas autorizadas;

advertencias;

reducciones aplicadas;

factor limitante;

request_id;

execution_id;

estado final;

evidence_ids;

event_ids;

receipt_hash.

El ContextReceipt deberá coincidir con:

sesión;

WorkspaceState;

ContextEnvelope;

ejecución real;

telemetría del runtime.

Un clic, un envío, un temporizador o una respuesta HTTP no podrán marcar un paso como done sin un receipt validado.

25. CICLO DE VIDA DE LOS RECEIPTS

Un receipt podrá tener estos estados:

emitted;

validated;

contradicted;

certified;

invalidated.

25.1. Emitted

El ejecutor ha producido el receipt.

No demuestra por sí mismo que sus afirmaciones sean correctas.

25.2. Validated

Un validador ha comprobado:

esquema;

identidad;

hashes;

coherencia;

efectos observables disponibles.

25.3. Contradicted

Existe evidencia incompatible con el receipt.

25.4. Certified

El receipt forma parte de una certificación independiente.

25.5. Invalidated

Cambió una condición material o se detectó una inconsistencia posterior.

Un receipt emitido no equivaldrá automáticamente a un efecto verificado.

26. RECUPERACIÓN DEL WORKSPACE

La recuperación operará dentro de:

`workspace_scope_root`

Normalmente:

`workspace_scope_root = project_root`

La recuperación no se limitará a `.gabo`, porque el código del proyecto puede vivir fuera de esa carpeta.

No podrá leer fuera de `workspace_scope_root` sin autorización explícita.

Cada fragmento registrará:

ruta absoluta normalizada;

ruta relativa respecto a `project_root`;

`workspace_id`;

`project_root`;

`workspace_scope_root`;

hash o revisión del archivo;

posición;

método;

puntuación;

estado epistemológico;

`request_id`;

`execution_id`.

El framework no podrá utilizar sus propios archivos como evidencia del proyecto salvo que:

el workspace activo sea BAGO;

la tarea trate expresamente sobre el framework;

el test lo autorice.

Antes de utilizar un fragmento deberá comprobarse:

que el archivo existe;

que pertenece al workspace;

que está dentro del alcance;

que no procede de una sesión anterior;

que su revisión no está obsoleta;

que no está desplazado por una fuente canónica más reciente.

Una ruta fuera del alcance deberá rechazarse y registrarse.

27. MOTOR DE INTENCIONES

El motor no podrá presentarse como aprendizaje o generalización cuando dependa principalmente de:

coincidencia exacta;

substrings;

palabras clave;

evaluación del dataset contra sí mismo.

El entrenamiento deberá separar:

entrenamiento;

validación;

prueba.

Las pruebas deberán incluir:

frases nuevas;

paráfrasis;

sinónimos;

negaciones;

ambigüedad;

casos fuera de distribución;

intenciones cercanas;

mensajes que deben continuar como chat.

Se medirán:

aciertos;

falsos positivos;

falsos negativos;

cobertura;

confianza;

casos no clasificados.

Una intención insegura no ejecutará automáticamente un comando.

BAGO podrá:

pedir confirmación;

continuar como conversación;

declarar intención desconocida.

La evolución solo podrá declararse cuando exista un cambio:

real;

persistido;

medible;

reversible;

comparado contra una referencia.

28. MEDICIÓN DEL CONTEXTO

BAGO diferenciará:

contexto declarado;

contexto posicional;

contexto permitido por adapter;

contexto permitido por runtime;

contexto configurado;

contexto aplicado;

contexto máximo verificado por memoria;

contexto cognitivo certificado;

contexto de trabajo;

contexto ocupado;

contexto disponible.

Cada dato se clasificará como:

observed;

calculated;

estimated;

declared;

certified;

unavailable.

Una estimación nunca se mostrará como observación.

29. LÍMITE TÉCNICO

La fórmula será:

$C_{\text{técnico}} = \text{mínimo entre:}$

$C_{\text{modelo}};$

$C_{\text{posicional}};$

$C_{\text{adapter}};$

$C_{\text{runtime}};$

$C_{\text{configurado}};$

$C_{\text{aplicado}}.$

El espacio técnico de entrada será:

$C_{\text{entrada_técnica}} = C_{\text{técnico}} \text{ menos } T_{\text{sistema}} \text{ menos } T_{\text{herramientas}} \text{ menos } T_{\text{salida_reservada}} \text{ menos } T_{\text{especiales}}$

Los tokens deberán contarse con el tokenizador exacto.

No se utilizarán aproximaciones por palabras o caracteres como resultado certificado.

30. MEMORIA Y CACHÉ KV

La dimensión de cada cabeza será:

$D_{\text{cabeza}} = H_{\text{oculto}} \text{ dividido entre } N_{\text{cabezas_atención}}$

La memoria teórica KV por token será:

$M_{\text{KV_por_token}} = L \text{ multiplicado por } N_{\text{cabezas_KV}} \text{ multiplicado por } D_{\text{cabeza}} \text{ multiplicado por la suma de } B_{\text{claves}} \text{ y } B_{\text{valores}}$

Cuando claves y valores utilicen el mismo tipo:

$M_{\text{KV_por_token}} = 2 \text{ multiplicado por } L \text{ multiplicado por } N_{\text{cabezas_KV}} \text{ multiplicado por } D_{\text{cabeza}} \text{ multiplicado por } B_{\text{KV}}$

El paralelismo solo se multiplicará cuando se confirme que cada secuencia conserva una caché independiente.

RAM y VRAM se medirán por separado.

La estimación teórica será:

$C_{\text{memoria_teórico}} = \text{parte entera de } M_{\text{disponible_KV}} \text{ dividido entre } M_{\text{KV_por_token}}$

Esta cifra será predictiva, no certificada.

31. LÍMITE REAL DE MEMORIA

Un contexto solo será válido cuando:

el modelo cargue;

el runtime aplique el valor solicitado;

el prefill termine;

se genere la salida reservada;

no exista error de memoria;

no exista truncamiento;

no exista reducción silenciosa;

la paginación permanezca dentro del umbral;

la caché se sitúe donde corresponde;

el proceso permanezca estable;

el resultado sea reproducible.

La medición incluirá:

sondeo progresivo;

valor válido;

valor fallido;

búsqueda binaria;

arranque frío;

arranque caliente;

generación corta;

generación máxima;

presión controlada;

paralelismo;

repeticiones.

Se registrarán:

RAM;

VRAM;

paginación;

contexto solicitado;

contexto aplicado;

prefill;

generación;

código de salida;

logs;

errores.

32. LÍMITE COGNITIVO

El benchmark medirá:

seguimiento de instrucciones;

recuperación;

razonamiento;

contradicciones;

herramientas;

código;

continuidad;

fidelidad a evidencia;

resistencia a inventar.

La puntuación podrá expresarse como:

$S(C) = wI \text{ multiplicado por } SI(C) \text{ más } wR \text{ multiplicado por } SR(C) \text{ más } wJ \text{ multiplicado por } SJ(C) \text{ más } wC \text{ multiplicado por } SC(C) \text{ más } wH \text{ multiplicado por } SH(C)$

Los pesos sumarán uno.

La información crítica se probará en:

cero por ciento;

veinticinco por ciento;

cincuenta por ciento;

setenta y cinco por ciento;

cien por ciento.

La puntuación robusta será la peor posición.

El límite inferior de confianza será:

$S_{inferior}(C) = media(C) - 1,96 \times \frac{desviación\ estándar(C)}{\sqrt{N}}$

No se seleccionará el mejor intento.

No se certificará con una sola semilla.

Una media no podrá ocultar un fallo crítico.

33. RESERVA OPERATIVA

La reserva derivará de telemetría real.

Considerará:

tokens de salida;

tokens de herramientas;

errores;

reintentos;

crecimiento de sesión;

resúmenes;

evidencias;

respuestas externas.

La fórmula será:

$R_{operativo}(tipo) = \text{percentil noventa y nueve de:}$

$T_{salida} + T_{herramientas} + T_{reintentos} + T_{crecimiento} + T_{evidencias}$

Se diferenciarán al menos:

consulta;

auditoría;

modificación de código;

diagnóstico;

uso intensivo de herramientas;

sesión autónoma prolongada.

No se utilizará un porcentaje fijo como valor certificado.

34. CONTEXTO DE TRABAJO

La fórmula será:

$C_{\text{trabajo}}(\text{tipo}) = \min(C_{\text{técnico}}, C_{\text{memoria_certificado}}, C_{\text{cognitivo_certificado}}(\text{tipo})) \text{ menos } R_{\text{operativo}}(\text{tipo})$

El contexto disponible será:

$C_{\text{disponible_ahora}} = C_{\text{trabajo}} \text{ menos } C_{\text{ocupado_actual}}$

BAGO mostrará por separado:

máximo certificado;

contexto de trabajo;

contexto ocupado;

contexto disponible;

factor limitante.

35. PRIORIDAD Y ADMISIÓN DEL CONTEXTO

35.1. Prioridades

Prioridad cero:

seguridad, identidad y restricciones inmutables.

Prioridad uno:

objetivo, criterios de aceptación y decisiones.

Prioridad dos:

evidencias, archivos y resultados relevantes.

Prioridad tres:

historial resumido.

Prioridad cuatro:

información auxiliar y recuperable.

Cuando falte espacio, BAGO eliminará primero prioridad cuatro.

No eliminará silenciosamente prioridades cero o uno.

35.2. Controlador de admisión

Antes de añadir contenido:

$C_{\text{proyectado}} = C_{\text{actual}} + T_{\text{nuevo}} + R_{\text{operativo}}$

Si supera C_{trabajo} , BAGO deberá:

resumir contenido prescindible;

recuperar solo fragmentos relevantes;

guardar externamente;

abrir una nueva fase;

rechazar contenido cuando no pueda preservar seguridad y objetivo.

Nunca truncará silenciosamente.

Toda decisión quedará registrada.

36. ARQUITECTURA CANÓNICA DE SUPERFICIES

BAGO deberá separar:

núcleo canónico;

presentadores;

transportes;

representaciones visuales.

La arquitectura lógica será:

Canonical Backend State

más TerminalPresenter

más ReactPresenter

más otros presentadores futuros.

Terminal y React podrán utilizar transportes distintos.

No podrán utilizar autoridades distintas.

36.1. Terminal real

La Terminal real será la experiencia REPL operada por el componente canónico de terminal.

Deberá consumir:

estado canónico;

registro canónico de comandos;

MenuState canónico;

CommandResult canónico;

receipts canónicos.

36.2. React

React será una superficie visual que consume DTOs canónicos mediante la API.

React no reimplementará:

autoridad de rutas;

binding;

menú operacional;

estado del pipeline;

claims;

modelo efectivo;

certificación.

36.3. Consola React

Una vista React que envíe comandos mediante HTTP no será denominada Terminal real salvo que encapsule verdaderamente el REPL.

Si solo utiliza la API, deberá denominarse:

Consola API;

Consola BAGO;

u otra denominación que no sugiera que es el REPL real.

36.4. Canal de origen

El canal podrá indicar que una petición procede de React o Terminal.

No podrá crear contratos operativos incompatibles.

Una misma acción canónica deberá producir:

el mismo command_id;

el mismo resultado semántico;

el mismo execution_id cuando forme parte de la misma ejecución;

el mismo estado final;

receipts equivalentes.

Solo podrá variar la representación visual.

37. FUENTE CANÓNICA DE CONTRATOS

BAGO deberá disponer de una única fuente de contratos de frontera.

La ubicación física podrá adaptarse a la arquitectura real, pero deberá existir una autoridad equivalente a:

framework_root/contracts/ui/v1/

Deberá contener esquemas para:

workspace-state;

session-status;

welcome-state;

menu-state;

command-result;

model-catalog;

execution-state;

pipeline-state;

claim-state;

context-envelope;

context-receipt;

error-state;

stream-event;

contradiction-record;

capability;

evidence-record;

receipt-state.

Los esquemas deberán ser:

versionados;

validables;

utilizados por productores;

utilizados por consumidores;

cubiertos por tests.

No deberán mantenerse definiciones manuales independientes en Python y TypeScript.

38. GENERACIÓN Y VALIDACIÓN DE TIPOS

Desde la fuente canónica deberán generarse o validarse:

modelos Python;

validadores Python;

tipos TypeScript;

validadores runtime del frontend;

fixtures;

tests de compatibilidad.

Un cambio contractual deberá actualizar la fuente canónica.

No se corregirá una incompatibilidad modificando únicamente un consumidor.

La integración continua deberá comprobar que:

los tipos generados están actualizados;

los fixtures validan;

el backend produce payloads válidos;

React acepta esos payloads;

Terminal interpreta el mismo estado semántico.

39. VERSIONADO DE CONTRATOS

Toda respuesta estructurada relevante deberá incluir:

contract_version;

producer;

request_id;

data;

warnings;

errors.

Cuando corresponda también incluirá:

execution_id;

receipt_id;

context_revision;

event_chain_head.

39.1. Compatibilidad mayor

Una diferencia incompatible en la versión mayor deberá bloquear el consumo.

La UI mostrará:

contract_incompatible.

No intentará interpretar el payload parcialmente.

39.2. Compatibilidad menor

Una diferencia menor solo podrá aceptarse cuando sea compatible hacia atrás.

Los campos desconocidos podrán ignorarse únicamente si el esquema lo permite.

Los campos obligatorios ausentes producirán error.

39.3. Negociación

El cliente podrá declarar las versiones que soporta.

El backend devolverá una versión compatible o un error estructurado.

39.4. Fuente de verdad

`contract_version` no será definido independientemente por React.

Procederá del backend y de los esquemas canónicos.

40. DTOS CANÓNICOS

40.1. WorkspaceState

Deberá contener:

estado;

raíces;

workspace_id;

session_id;

manifiesto;

binding;

razón;

allowed_actions;

blocked_operations;

context_revision;

warnings;

contradiction_ids;

contract_version.

40.2. SessionStatus

Deberá tener una forma única.

No duplicará ambigualmente los mismos campos en distintos niveles.

Contendrá como mínimo:

session;

workspace;

runtime;

context;

permissions;

execution;

contradictions;

contract_version.

Todos los consumidores deberán utilizar la misma estructura.

40.3. MenuState

MenuState representará la semántica completa del menú operacional producida por el backend.

No impondrá el dibujo concreto, la distribución de columnas, la navegación por teclado ni la apariencia visual de Terminal o React.

MenuState contendrá:

contract_version;

workspace_state;

centros_operativos;

centro_activo;

secciones_visibles;

elemento_seleccionado;

estado_de_seleccion;

resumen_de_estado;

acciones_recomendadas;

acciones_permitidas;

acciones_secundarias;

acciones_bloqueadas;

razones_de_bloqueo;

comandos_directos_disponibles;

operaciones_activas;

advertencias;

contradiction_ids;

navegacion;

request_id.

40.3.1. Centros operativos

Cada centro deberá incluir:

center_id;

nombre;

descripción;

estado;

resumen;

selección_actual;

cantidad_de_elementos;

acciones_visibles;

acciones_bloqueadas;

advertencias;

actividad_reciente;

profundidad_disponible.

40.3.2. Centro activo

`centro_activo` identificará el dominio representado actualmente.

La activación visual de un centro no modificará por sí sola el estado operacional.

40.3.3. Selección

`elemento_seleccionado` podrá representar la selección visual actual.

`estado_de_seleccion` distinguirá:

visual;

pending_confirmation;

effective;

rejected;

stale.

Una selección visual no será efectiva hasta que el backend la valide y confirme.

40.3.4. Acciones

MenuState distinguirá:

acciones_recomendadas;

acciones_permitidas;

acciones_secundarias;

acciones_bloqueadas.

Cada acción deberá referirse a un command_id canónico.

Cada acción bloqueada deberá incluir una razón estructurada.

40.3.5. Comandos directos

`comandos_directos_disponibles` contendrá las invocaciones rápidas aplicables al estado actual.

La existencia de un comando directo no obligará a mostrarlo como rama permanente.

40.3.6. Navegación

`navegacion` podrá contener:

nivel_actual;

ruta_visual;

nivel_anterior;

puede_volver;

puede_buscar;

puede_filtrar;

paginacion;

filtros_activos.

La navegación no será una autoridad alternativa.

40.4. ModelCatalog

Deberá utilizar una forma única:

provider;

items;

selected_model;

effective_model;

capabilities;

evidence_state;

contract_version.

No existirán claves alternativas para representar la misma lista.

40.5. CommandResult

Deberá contener:

ok;

command_id;

message;

data;

execution_state;

request_id;

execution_id;

receipt_ids;

warnings;

errors;

contradiction_ids;

contract_version.

40.6. PipelineState

Deberá contener:

pipeline_id;

execution_id;

steps;

transitions;

evidence_ids;

receipt_ids;

current_state;

blocked_reason;

contract_version.

40.7. ClaimState

Deberá contener:

claim_id;

statement;

criticality;

acceptance_condition;

test_ids;

evidence_ids;

contradiction_ids;

execution_state;

maturity_state;

certification_state;

fingerprint;

contract_version.

40.8. ErrorState

Deberá contener:

code;

message;

cause;

details;

missing_requirements;

allowed_actions;

request_id;

execution_id;

component_id;

contract_version.

40.9. WelcomeState

Deberá contener:

workspace_state;

identity;

summary;

recommended_actions;

allowed_actions;

blocked_operations;

warnings;

contradiction_ids;

contract_version.

40.10. ContradictionRecord

Deberá contener:

contradiction_id;

subject;

source_a;

value_a;

source_b;

value_b;

criticality;

detected_at;

affected_claims;

affected_operations;

resolution_state;

resolution_evidence;

contract_version.

40.11. Capability

Deberá contener:

capability_id;

capability_type;

scope;

workspace_id;

session_id;

execution_id;

allowed_effects;

issued_at;

expires_at;

issuer;

revocation_state;

nonce;

conditions;

contract_version.

40.12. EvidenceRecord

Deberá contener:

evidence_id;

type;

origin;

hash;

created_at;

valid_until;

producer;

workspace_id;

execution_id;

claim_ids;

integrity_state;

epistemic_state;

contract_version.

41. CONTRATO DE API Y TRANSPORTE

La API deberá funcionar como transporte del estado canónico.

No será una segunda autoridad.

Cada handler deberá:

recibir una solicitud validada;

resolver el estado canónico;

ejecutar el servicio correspondiente;

producir un DTO validado;

devolver errores estructurados.

No devolverá diccionarios ambiguos sin contrato.

41.1. Recursos mínimos

La API deberá exponer de manera contractual capacidades equivalentes a:

estado de sesión;

estado de workspace;

menú;

catálogo de modelos;

chat;

comandos;

cambio de modelo;

pipeline;

claims;

receipts;

evidencias;

contradicciones;

capabilities.

Los nombres físicos de los endpoints podrán adaptarse.

La semántica permanecerá canónica.

41.2. Transporte y ejecución

Un HTTP correcto no implicará necesariamente que una operación esté done.

La respuesta deberá distinguir:

éxito del transporte;

estado de ejecución;

estado de validación;

estado de certificación.

41.3. Errores

Los errores no deberán ocultarse dentro de una respuesta aparentemente correcta.

Deberán utilizar ErrorState.

42. EVENTOS Y STREAMING

Las operaciones progresivas deberán emitir eventos estructurados.

Los eventos canónicos podrán incluir:

execution.proposed;

execution.validated;

execution.authorized;

execution.started;

execution.progress;

pipeline.transition;

model.token.delta;

tool.started;

tool.completed;

tool.failed;

receipt.created;

receipt.validated;

contradiction.detected;

execution.completed;

execution.failed;

execution.blocked;

execution.cancelled.

Cada evento deberá contener:

contract_version;

sequence;

request_id;

execution_id;

session_id;

workspace_id;

component_id;

timestamp;

event_type;

payload_hash;

previous_event_hash;

event_hash;

actor;

payload.

React y Terminal deberán poder reconstruir el mismo estado final a partir de los eventos.

Ninguna UI marcará una operación como completada solo porque haya dejado de recibir eventos.

43. LOG DE EVENTOS CON INTEGRIDAD VERIFICABLE

Los eventos relevantes se almacenarán de forma append-only.

Cada evento enlazará con el hash anterior.

La cadena deberá permitir detectar:

modificación;

eliminación;

reordenación;

inserción no autorizada.

43.1. Rotación

Los logs podrán rotarse por:

tamaño;

tiempo;

workspace;

sesión.

Cada segmento deberá registrar:

hash inicial;

hash final;

segmento anterior;

segmento siguiente.

43.2. Concurrencia

La escritura concurrente deberá preservar:

secuencia monotónica;

orden estable;

ausencia de colisiones.

43.3. Recuperación

Una corrupción parcial deberá:

marcar el segmento como degraded;

identificar el último evento válido;

bloquear certificaciones dependientes;

permitir diagnóstico.

43.4. Retención

La política deberá definir:

tiempo de conservación;

eventos obligatorios;

eventos resumibles;

eventos eliminables;

protección de datos.

43.5. Privacidad

Los payloads sensibles podrán almacenarse mediante:

hash;

referencia;

redacción;

cifrado.

La integridad deberá preservarse sin exponer secretos.

44. REGISTRO CANÓNICO DE COMANDOS Y MENÚ

Deberá existir un único registro canónico de comandos.

El registro definirá:

command_id;

nombre;

categoría;

descripción;

argumentos;

tipos;

permisos;

capabilities;

precondiciones;

estados en los que está permitido;

estado experimental;

efectos;

necesidad de confirmación;

servicio responsable;

tipo de receipt;

eventos esperados;

reversión;

nivel de riesgo.

El backend generará MenuState utilizando:

registro de comandos;

WorkspaceState;

sesión;

permisos;

modo;

capacidades disponibles;

ejecuciones activas.

Terminal y React renderizarán MenuState.

No mantendrán listas paralelas de comandos operativos.

Una UI podrá añadir accesos visuales únicamente cuando apunten a un command_id canónico existente.

45. CENTROS OPERATIVOS DEL MENÚ

El menú  no representará el registro canónico como una lista plana de todas las operaciones disponibles.

Las entradas principales actuarán como centros operativos completos.

Cada centro reunirá:

estado actual;

resumen;

elementos registrados;

elementos disponibles;

elemento seleccionado;

acciones recomendadas;

acciones permitidas;

acciones secundarias;

acciones bloqueadas;

razones de bloqueo;

advertencias;

actividad reciente;

acceso a detalles.

Las operaciones de visualizar, listar, registrar, inspeccionar y seleccionar no deberán convertirse automáticamente en ramas independientes cuando puedan resolverse claramente dentro del mismo centro.

La regla será:

UNA ENTRADA PRINCIPAL POR DOMINIO OPERATIVO.

VISUALIZACIÓN, REGISTRO, LISTADO Y SELECCIÓN UNIFICADOS CUANDO LA CARGA LO PERMITA.

DESGLOSE PROGRESIVO SOLO CUANDO EXISTA COMPLEJIDAD REAL.

45.1. Dominios principales

El primer nivel del menú contendrá centros equivalentes a:

/task

/session

/workspace

/context

/model

/tools

/evidence

/system

/view

/advanced

Cada entrada abrirá el estado consolidado de su dominio.

No será una carpeta vacía cuya única función sea mostrar otra lista de comandos.

45.2. Contenido mínimo

Al abrir un centro, la Terminal mostrará cuando corresponda:

identidad del dominio;

estado confirmado;

selección efectiva;

selección visual pendiente;

resumen operativo;

colección principal;

acciones recomendadas;

acciones permitidas;

acciones secundarias;

acciones bloqueadas;

causas de bloqueo;

advertencias;

últimas operaciones.

45.3. Registro y árbol visible

REGISTRO CANÓNICO NO EQUIVALE A ÁRBOL VISIBLE.

El árbol visible no podrá inventar comandos.

El registro no obligará a mostrar todos los comandos.

No deberán mantenerse el registro y el árbol como listas manuales independientes.

45.4. Comandos directos

Toda acción relevante podrá conservar una invocación directa para:

usuarios avanzados;

automatización;

scripts;

tests;

documentación;

reproducción.

Los comandos directos deberán utilizar:

el mismo command_id;

el mismo servicio;

la misma validación;

los mismos permisos;

las mismas capabilities;

la misma confirmación;

el mismo tipo de receipt;

que la acción equivalente del centro operativo.

45.5. Navegación interactiva

TerminalPresenter podrá permitir:

mover selección;

buscar;

filtrar;

expandir;

contraer;

abrir detalles;

ejecutar una acción permitida;

volver;

cerrar el menú.

La autoridad sobre estado, selección efectiva, permisos, ejecución y resultado permanecerá en el backend.

45.6. Desglose por carga

Un centro solo se dividirá cuando exista:

paginación;

entidades claramente diferentes;

información excesiva;

permisos o riesgos diferentes;

operaciones destructivas;

ejecuciones prolongadas;

foco de selección independiente;

capacidad avanzada o experimental;

pérdida de comprensión;

aumento del riesgo.

No se crearán subniveles únicamente para reproducir:

status;

list;

show;

inspect;

select;

history;

register.

45.7. Profundidad

La profundidad ordinaria será:

primer nivel: dominio;

segundo nivel: sección necesaria;

tercer nivel: elemento o acción concreta.

No se añadirán niveles posteriores salvo necesidad demostrada.

45.8. Criterio de simplicidad

CENTRO OPERATIVO = ESTADO MÁS COLECCIÓN MÁS SELECCIÓN MÁS ACCIONES MÁS DETALLE CONTEXTUAL

SUBMENÚ NECESARIO = CARGA ELEVADA O ENTIDADES HETEROGÉNEAS O RIESGO DIFERENCIADO O EJECUCIÓN PROLONGADA O NECESIDAD REAL DE FOCO INDEPENDIENTE

46. BIENVENIDA ADAPTATIVA

La bienvenida no mostrará todas las acciones posibles.

Mostrará únicamente acciones coherentes con WelcomeState y allowed_actions.

46.1. Workspace válido y sesión vinculada

Mostrará:

framework;

proyecto;

workspace_id;

workspace_root;

sesión;

modelo;

contexto;

objetivo actual.

Ofrecerá:

continuar;

nueva tarea;

estado del workspace;

inspeccionar contexto;

cambiar de proyecto.

No ofrecerá inicialización.

46.2. Workspace detectado y sesión no vinculada

Mostrará:

proyecto;

workspace_id;

workspace_root;

manifiesto;

sesión no vinculada.

Ofrecerá:

vincular sesión;

ver detalles;

abrir sesión anterior;

seleccionar otro proyecto;

verificar workspace.

No ofrecerá inicialización.

46.3. Workspace existente pero incoherente

Mostrará:

raíces;

código de error;

causa;

operaciones bloqueadas.

Ofrecerá:

ver diferencias;

verificar;

reparar;

modo seguro;

seleccionar otro proyecto.

No ofrecerá inicialización automática.

46.4. Proyecto sin .gabo

Solo en este estado ofrecerá:

inicializar .gabo ;

vincular otro proyecto;

abrir sesión anterior;

examinar sin modificar;

diagnóstico.

46.5. .bago legacy sin .gabo

Ofrecerá:

inspeccionar legacy;

migrar;

crear .gabo sin importar;

solo lectura;

seleccionar otro proyecto.

46.6. Estado bloqueado

No abrirá un chat aparentemente normal.

Indicará:

qué autoridad no coincide;

qué operación está bloqueada;

si se llamó o no al modelo;

si se ejecutó alguna herramienta;

qué acciones permiten resolverlo.

47. PROMPT OPERACIONAL DE TERMINAL

El prompt será compacto y reflejará estado confirmado.

Ejemplos:

bago:BAGO [ctx:confirmed] ›

bago:BAGO [ctx:partial] ›

bago:BAGO [ctx:stale] ›

bago [workspace-detected] ›

bago [no-workspace] ›

bago:BAGO [blocked] ›

bago:BAGO [approval] ›

bago:BAGO [contradicted] ›

El prompt no será autoridad.

Representará el estado recibido del backend.

48. CENTRO OPERATIVO /TASK

El centro /task reunirá:

objetivo actual;

criterios de aceptación;

decisiones;

restricciones;

estado;

siguiente paso;

plan vigente;

ejecuciones activas;

historial reciente;

acciones permitidas.

Desde la superficie se podrá:

crear una tarea;

continuar;

planificar;

ejecutar;

aprobar;

cancelar;

reintentar;

consultar historial.

El registro podrá exponer comandos directos equivalentes a:

/task status

/task new

/task continue

/task plan

/task run

/task approve

/task cancel

/task retry

49. CENTRO OPERATIVO /SESSION

El centro /session reunirá:

sesión activa;

workspace vinculado;

objetivo;

decisiones;

restricciones;

context_revision;

persistencia;

sesiones recientes;

sesión seleccionada;

acciones permitidas;

acciones bloqueadas.

Desde la superficie se podrá:

crear una sesión;

seleccionar una sesión;

reanudar;

guardar;

renombrar;

cerrar;

exportar.

El registro podrá exponer:

/session status

/session new

/session list

/session resume

/session save

/session rename

/session close

/session export

50. CENTRO OPERATIVO **/WORKSPACE**

El centro **/workspace** reunirá:

WorkspaceState;

project_root;

workspace_root;

workspace_scope_root;

workspace_id;

estado del manifiesto;

vinculación;

workspace actual;

workspaces detectados;

workspace seleccionado;

acciones recomendadas;

acciones permitidas;

acciones bloqueadas;

advertencias.

Desde la superficie se podrá:

seleccionar un workspace;

vincular la sesión;

cambiar proyecto;

verificar;

reparar;

reindexar;

inspeccionar legacy;

migrar;

desvincular.

La inicialización solo aparecerá cuando:

WorkspaceState permita workspace.init;

`.gabo` no exista;

el backend incluya la acción en `allowed_actions`.

El registro podrá exponer:

`/workspace status`

`/workspace roots`

`/workspace detect`

`/workspace link`

`/workspace switch`

`/workspace init`

`/workspace verify`

`/workspace repair`

`/workspace scope`

`/workspace reindex`

/workspace inspect-legacy

/workspace migrate-legacy

/workspace unlink

51. CENTRO OPERATIVO /CONTEXT

El centro `/context` reunirá:

estado del contexto;

ContextEnvelope vigente;

último ContextReceipt;

context_revision;

presupuesto;

contexto ocupado;

contexto disponible;

reserva;

factor limitante;

archivos representados;

fragmentos recuperados;

reducciones;

advertencias;

mediciones recientes.

Desde la superficie se podrá:

inspeccionar;

reconstruir;

medir;

ejecutar benchmark;

certificar;

calibrar;

invalidar;

consultar historial.

El registro podrá exponer:

/context status

/context inspect

/context envelope

/context receipt

/context budget

/context rebuild

/context measure

/context benchmark

/context certify

/context calibrate

/context history

/context invalidate

/context explain

52. CENTRO OPERATIVO /MODEL

El centro /model reunirá:

provider efectivo;

adapter efectivo;

runtime efectivo;

modelo configurado;

modelo efectivo;

estado;

catálogo;

capacidades;

modelo seleccionado;

acciones permitidas;

acciones bloqueadas;

historial.

Desde la superficie se podrá:

seleccionar modelo;

inspeccionar capacidades;

probar;

activar selección automática;

consultar historial.

El registro podrá exponer:

/model status

/model list

/model select

/model capabilities

/model test

/model auto

/model history

No existirá más de un selector efectivo de modelo.

53. CENTRO OPERATIVO /TOOLS

El centro `/tools` reunirá:

registro de herramientas;

herramientas disponibles;

herramientas autorizadas;

herramientas bloqueadas;

herramienta seleccionada;

permisos requeridos;

ejecuciones activas;

historial reciente.

Desde la superficie se podrá:

seleccionar;

consultar argumentos;

autorizar;

denegar;

ejecutar;

cancelar;

inspeccionar resultados.

La autorización y ejecución permanecerán bajo control del backend.

El registro podrá exponer:

`/tools status`

`/tools list`

`/tools allowed`

`/tools allow`

/tools deny

/tools run

/tools cancel

/tools history

54. CENTRO OPERATIVO /EVIDENCE

El centro /evidence reunirá:

claims;

receipts;

logs;

diffs;

tests;

mutaciones;

contradicciones;

claim seleccionado;

receipt seleccionado;

estado de integridad;

estado de madurez;

estado de certificación;

acciones permitidas.

Desde la superficie se podrá:

filtrar;

seleccionar;

inspeccionar;

comparar;

verificar integridad;

exportar;

consultar pruebas.

El registro podrá exponer:

/evidence latest

/evidence claims

/evidence claim

/evidence receipts

/evidence receipt

/evidence logs

/evidence contradictions

/evidence verify

/evidence diff

/evidence tests

/evidence mutations

/evidence export

55. CENTRO OPERATIVO /SYSTEM

El centro /system reunirá:

framework_version;

framework_root;

estado del backend;

bridge;

configuración;

logs;

integridad de eventos;

modo seguro;

servicios;

diagnóstico;

acciones permitidas.

Desde la superficie se podrá:

consultar estado;

ejecutar doctor;

ver versión;

inspeccionar raíces;

ver configuración;

inspeccionar bridge;

consultar logs;

verificar cadena;

activar modo seguro;

reiniciar.

El registro podrá exponer:

/system status

/system doctor

/system version

/system roots

/system config

/system bridge

/system logs

/system verify-chain

/system safe-mode

/system restart

56. CENTRO OPERATIVO **/VIEW**

El centro **/view** reunirá preferencias exclusivamente visuales:

densidad;

paneles;

modo compacto;

modo normal;

modo detallado;

estado de presentación.

Desde la superficie se podrá:

cambiar densidad;

abrir o cerrar paneles;

limpiar presentación;

restablecer vista.

El registro podrá exponer:

/view compact

/view normal

/view verbose

/view status

/view panels

/view clear

La vista no modificará estado operacional.

57. CENTRO OPERATIVO /ADVANCED

El centro /advanced reunirá capacidades:

experimentales;

internas;

de diagnóstico;

de benchmark;

de intención;

de certificación;

de migración avanzada;

de integridad;

de pruebas adversariales.

El registro podrá exponer:

/advanced intent status

/advanced intent evaluate

/advanced intent test-novel

/advanced intent evolve

/advanced certification fingerprint

/advanced legacy scan

/advanced benchmark datasets

/advanced integrity verify

/advanced contradictions inspect

/advanced adversarial run

58. AUTORIDAD DE LA INTERFAZ

El chat seguirá siendo el centro.

La UI representará únicamente estados confirmados por el backend.

La UI no podrá declarar localmente:

workspace confirmado;

modelo efectivo;

provider efectivo;

pipeline done;

claim verificado;

capacidad certificada;

política aplicada;

simulación ejecutada;

selección efectiva;

receipt validado;

integridad confirmada.

Mientras no exista confirmación utilizará:

unknown;

loading;

degraded;

error;

stale;

contradicted.

58.1. LocalStorage

LocalStorage solo podrá conservar:

tema;

densidad visual;

paneles abiertos;

tamaño de componentes;

preferencias no operativas.

No podrá ser autoridad para:

workspace;

sesión;

modelo;

provider;

pipeline;

claims;

permisos;

certificación;

context_revision;

selección efectiva;

allowed_actions;

receipts;

contradicciones.

58.2. Acciones de usuario

La UI podrá emitir:

intención;

solicitud;

selección visual;

confirmación;

cancelación.

El backend devolverá:

selección efectiva;

estado de ejecución;

estado de madurez;

estado de certificación;

evidencias;

receipts;

contradicciones.

59. PROHIBICIÓN DE DEFAULTS ENGAÑOSOS

No se utilizarán valores predeterminados que aparenten un estado real.

Ejemplos prohibidos:

modelo efectivo predeterminado;

workspace confirmado predeterminado;

instalación ready predeterminada;

pipeline activo predeterminado;

claim ok predeterminado;

provider operativo predeterminado;

menú permitido predeterminado.

Los valores iniciales serán:

unknown;

loading;

not_available;

unbound;

pending_validation.

60. PROHIBICIÓN DE EXCEPCIONES SILENCIOSAS

Una excepción no podrá convertirse en:

lista vacía;

objeto vacío;

estado ready;

done;

respuesta correcta sin warnings.

Toda excepción relevante deberá producir:

ErrorState;

evento;

log;

request_id;

component_id;

impacto;

acciones permitidas.

Capturar una excepción y continuar solo será válido cuando el contrato defina explícitamente una degradación segura.

61. PIPELINE

Estados:

pending;

validated;

authorized;

running;

done;

failed;

blocked;

cancelled.

No existirá transición directa a done sin ejecución y validación.

Un paso no pasará a done si el workspace no está confirmado.

Cada transición será autorizada por el backend.

La UI no podrá marcar un nodo como completado.

Done exigirá:

execution_id;

resultado validado;

receipt validado;

evidence_ids;

invariantes satisfechos.

Blocked incluirá causa estructurada.

Failed incluirá:

error;

evidencia;

punto de fallo.

Cancelled no podrá representarse como done.

62. CLAIMS

Solo el backend podrá registrar o modificar ClaimState canónico.

La UI podrá crear:

borradores;

observaciones;

solicitudes de revisión.

Verified exigirá:

acceptance_condition satisfecha;

tests ejecutados;

evidence_ids;

ausencia de contradicciones críticas.

Certified exigirá además:

certificador independiente;

fingerprint;

repeticiones;

pruebas adversariales;

CertificationReceipt.

63. AUTORIDAD DEL MODELO EFECTIVO

BAGO distinguirá:

modelo solicitado;

modelo configurado;

modelo seleccionado;

modelo cargado;

modelo efectivo;

modelo mostrado.

El modelo efectivo solo podrá proceder de:

telemetría del runtime;

respuesta verificable del provider;

ModelExecutionReceipt validado.

La interfaz no podrá asumir que el modelo solicitado fue el utilizado.

64. AUTORIDAD DE HERRAMIENTAS

Una herramienta solo podrá considerarse ejecutada cuando exista:

ToolReceipt;

execution_id;

capability válida;

efecto observado o salida validada;

estado final.

La afirmación del modelo “he ejecutado la herramienta” no será suficiente.

La UI no podrá simular herramientas como completadas.

65. AUTORIDAD DE ARCHIVOS Y MODIFICACIONES

Toda modificación de archivos deberá registrar:

ruta normalizada;

workspace_id;

scope;

hash anterior;

hash posterior;

operación;

actor;

execution_id;

capability_id;

backup o estrategia de reversión;

validación posterior.

Una modificación no se considerará aplicada hasta comprobar el archivo resultante.

66. MODELO DE CAPABILITIES

Las operaciones con efecto requerirán capabilities explícitas.

Ejemplos:

read_workspace;

write_workspace;

execute_tool;

change_model;

bind_workspace;

modify_pipeline;

register_claim;

certify_claim;

migrate_legacy;

change_configuration.

66.1. Emisión

Solo un servicio autorizado podrá emitir capabilities.

El issuer deberá estar identificado.

66.2. Validación

Antes de ejecutar deberá comprobarse:

firma o integridad;

issuer;

scope;

workspace_id;

session_id;

execution_id;

fecha;

nonce;

condiciones;

revocación.

66.3. Caducidad

Toda capability tendrá:

expires_at;

o una condición equivalente de finalización.

66.4. Revocación

Deberá existir un registro de revocaciones.

Una capability revocada no podrá reutilizarse.

66.5. Prevención de reutilización

Las capabilities de un solo uso deberán incluir:

nonce;

execution_id;

consumed_at.

66.6. Privilegio mínimo

Una capability solo concederá los efectos necesarios para la operación concreta.

La visibilidad de una acción no implicará capability.

67. EJECUCIONES PROLONGADAS

Todo trabajo prolongado deberá disponer de:

execution_id;

timeout;

cancelación;

reintento controlado;

idempotency_key cuando sea necesario;

estado persistido;

punto de recuperación;

limpieza posterior;

receipt final.

Un reintento no deberá duplicar:

modificaciones;

mensajes;

trabajos;

cambios de configuración;

evidencias.

Una cancelación deberá terminar como cancelled o failed con razón equivalente.

Nunca como done.

68. TESTIGO OPERACIONAL

Las operaciones críticas deberán disponer de uno o varios testigos cuando sea razonablemente posible.

El testigo observará:

solicitud;

preflight;

inicio;

efectos;

resultado;

receipt;

estado final.

Ejemplos:

filesystem watcher;

auditor de procesos;

telemetría de runtime;

verificador de persistencia;

comparador de hashes;

monitor de herramientas.

El testigo no deberá depender exclusivamente del mismo dato local producido por el ejecutor.

69. PRINCIPIO DE DOBLE FUENTE

Los hechos críticos deberán confirmarse mediante dos fuentes cuando sea razonablemente posible.

Ejemplos:

modelo solicitado más modelo observado;

ruta declarada más ruta normalizada por filesystem;

escritura solicitada más hash posterior;

herramienta invocada más ToolReceipt;

contexto solicitado más contexto aplicado;

sesión guardada más recarga desde persistencia.

Dos fuentes derivadas del mismo valor no verificado no serán consideradas independientes.

70. CONTRADICTIONRECORD

Cuando dos fuentes relevantes discrepen, BAGO deberá crear un ContradictionRecord.

Estados:

open;

contained;

resolved;

accepted_exception.

Una contradicción crítica deberá:

bloquear operaciones afectadas;

invalidar estados dependientes;

aparecer en la interfaz;

conservar evidencias;

ofrecer acciones de resolución.

No se elegirá silenciosamente un valor.

71. MOTOR DE COHERENCIA

BAGO deberá disponer de un motor de coherencia que verifique invariantes.

Ejemplos:

workspace_root es project_root/.gabo;

workspace_id coincide con el manifiesto;

session.workspace_id coincide con WorkspaceState;

ContextEnvelope y ContextReceipt comparten identidad;

modelo mostrado coincide con modelo efectivo;

acción visible pertenece a allowed_actions;

acción visible tiene command_id;

pipeline done contiene receipt;

claim verified contiene evidencias;

certificación coincide con fingerprint;

tool result pertenece al execution_id correcto;

ruta de evidencia está dentro del scope;

receipt_hash es válido;

event_chain_head coincide.

El motor deberá ejecutarse:

antes de operar;

después de operar;

antes de persistir;

antes de presentar estados críticos;

antes de certificar.

72. CONFIANZA CERO ENTRE CAPAS

Ninguna capa confiará automáticamente en los datos de otra.

El backend validará solicitudes de UI.

La API validará payloads de entrada.

React validará DTOs recibidos.

Terminal validará DTOs recibidos.

El runtime será contrastado mediante telemetría.

Los receipts serán validados antes de utilizarse.

Las evidencias serán verificadas mediante hash o revisión.

La frase “proviene del backend” no será suficiente si el objeto no valida contra el contrato.

73. CADUCIDAD E INVALIDACIÓN DE EVIDENCIAS

Cada evidencia deberá indicar:

created_at;

valid_until cuando corresponda;

revision;

fingerprint;

invalidation_conditions.

Ejemplos:

un catálogo de modelos caduca cuando cambia el runtime;

un índice caduca cuando cambia un archivo;

un binding caduca cuando cambia el manifiesto;

una certificación caduca cuando cambia el código;

una selección efectiva caduca cuando desaparece el elemento.

Una evidencia caducada no podrá utilizarse como vigente.

74. CONTENCIÓN DE INCOHERENCIAS

Cuando se detecte una contradicción:

se bloquearán las operaciones afectadas;

no se bloquearán necesariamente dominios no relacionados;

se conservarán evidencias;

se marcará el estado como contradicted o degraded;

se ofrecerán acciones de resolución;

no se elegirá silenciosamente el valor más conveniente.

La contención deberá ser proporcional al alcance del fallo.

75. RESPONSABILIDAD POR COMPONENTE

Cada componente deberá declarar:

component_id;

versión;

contratos producidos;

contratos consumidos;

estados que puede modificar;

capabilities requeridas;

eventos emitidos;

receipts generados;

fallos posibles.

No se aceptarán componentes anónimos que modifiquen estado canónico.

76. SEGURIDAD DE SECRETOS

Ningún ContextEnvelope, ContextReceipt, log o paquete de evidencias conservará secretos en claro.

Deberán eliminarse o enmascarse:

tokens;

claves API;

cabeceras de autorización;

credenciales;

cookies;

variables sensibles;

contenido marcado como secreto.

La trazabilidad se conservará sin exponer el secreto.

77. CERTIFICATIONFINGERPRINT

Toda certificación se vinculará a una huella exacta.

Incluirá:

commit o hash del código;

hash de configuración;

framework_version;

contract_version;

framework_root;

workspace_id cuando corresponda;

hash del modelo;

tokenizador;

provider;

adapter;

runtime;

versión del runtime;

hardware;

contexto;

cuantización;

caché KV;

paralelismo;

batch;

plantilla;

prompt del sistema;

herramientas;

versión del benchmark;

hash del dataset;

versión de ContextEnvelope;

versión de ContextReceipt;

versión de los presenters;

versión del registro canónico;

versión de MenuState;

versión del motor de coherencia;

event_chain_head.

Dos huellas diferentes no compartirán automáticamente una certificación.

78. INVALIDACIÓN

Una certificación se invalidará cuando cambie materialmente:

código;

configuración;

framework;

contratos;

workspace;

modelo;

tokenizador;

provider;

adapter;

runtime;

hardware;

cuantización;

KV;

paralelismo;

batch;

plantilla;

prompt;

herramientas;

benchmark;

dataset;

esquemas;

presenters;

registro canónico;

semántica del menú;

motor de coherencia;

política de capabilities.

La causa quedará registrada.

Una certificación invalidada no podrá utilizarse como vigente.

79. PUERTAS DE ACEPTACIÓN

Cada fase tendrá una puerta de aceptación.

No se avanzará hasta completar:

prueba positiva;

prueba negativa;

prueba de error;

prueba de persistencia;

prueba de regresión;

prueba de contrato;

prueba de equivalencia;

prueba inversa;

prueba de mutación cuando corresponda.

El ciclo será:

requisito;

claim;

test inicialmente fallido;

implementación mínima;

test positivo;

test negativo;

test de error;

test de contrato;

test de equivalencia;

prueba inversa;

mutación;

validación;

receipt;

repetición desde entorno limpio.

80. PRUEBA DE INVERSIÓN

Toda garantía crítica deberá acompañarse de una prueba que demuestre que su incumplimiento es detectado.

Ejemplos:

si workspace_root se valida, debe probarse uno incorrecto;

si done exige receipt, debe probarse done sin receipt;

si el modelo efectivo coincide, debe probarse otro modelo;

si la cadena es íntegra, debe alterarse un evento.

La regla será:

NINGUNA GARANTÍA SIN PRUEBA DE QUE SU INCUMPLIMIENTO ES DETECTADO.

81. JERARQUÍA DE EVIDENCIAS

Orden de fiabilidad:

primero, efecto real observado;

segundo, testigo operacional independiente;

tercero, respuesta estructurada validada del backend;

cuarto, persistencia verificada;

quinto, logs del runtime;

sexto, tests automáticos;

séptimo, inspección de código;

octavo, interfaz;

noveno, explicación del implementador.

Una evidencia inferior no sustituirá una superior cuando esta sea necesaria.

82. CONDICIONES QUE BLOQUEAN LA CERTIFICACIÓN

No podrá certificarse una capacidad cuando exista:

fallo critical;

test crítico omitido;

ruta inventada;

ejecución simulada;

framework_root incoherente;

workspace inconsistente;

alcance no validado;

modelo diferente;

provider diferente;

runtime diferente;

ContextEnvelope ausente;

ContextReceipt ausente o incoherente;

estado visual sin respaldo;

contrato no validado;

productor y consumidor incompatibles;

verificación solo con mocks;

repeticiones insuficientes;

reducción silenciosa;

cambio sin reversión;

regresión relevante;

evidencia incompleta;

resultado no reproducible;

autocertificación;

certificación no invalidada;

secretos expuestos;

reintentos no idempotentes;

`.bago` legacy como autoridad;

duplicación de `.gabo`;

bienvenida incoherente;

menú incompatible con WorkspaceState;

estado operacional almacenado únicamente en frontend;

registro y árbol visible independientes;

selección visual aplicada como efectiva;

acción interactiva diferente del comando directo;

submenús sin justificación;

estado crítico construido sin validador;

receipt no validado;

cadena de eventos alterable sin detección;

capability excesiva o reutilizada;

contradicción crítica abierta;

default operativo engañoso;

excepción silenciada;

evidencia sin procedencia;

componente anónimo modificando estado.

El resultado será:

NOT_CERTIFIED

o:

REJECTED.

83. ARQUITECTURA DE PRUEBAS

Las pruebas deberán separarse en:

unitarias;

contractuales;

integración;

extremo a extremo;

adversariales;

mutación;

rendimiento;

certificación;

coherencia de interfaz;

equivalencia de presentadores;

integridad de eventos;

capabilities;

resistencia a la falsedad.

83.1. Suite mantenida

Deberá existir una ruta o configuración inequívoca para los tests mantenidos.

La recogida no incluirá accidentalmente:

scripts históricos;

one-off tests;

pruebas de migraciones antiguas;

scripts que ejecutan acciones al importarse.

83.2. Pruebas históricas

Las pruebas de una sola ejecución deberán:

aislarse;

documentarse;

no recogerse automáticamente;

no considerarse certificación.

83.3. Portabilidad

Los tests no utilizarán rutas absolutas personales.

Deberán utilizar:

temporales;

fixtures;

variables controladas;

rutas relativas;

abstracciones multiplataforma.

83.4. Sistemas soportados

La suite deberá ejecutarse en los sistemas operativos soportados.

83.5. Self-tests

Los self-tests documentados deberán formar parte de la regresión mantenida.

No podrán permanecer recomendados si no se ejecutan en integración continua.

84. PRUEBAS DE CONTRATO ENTRE PRODUCTOR Y CONSUMIDOR

Cada DTO deberá tener:

fixtures válidos;

fixtures inválidos;

test de producción backend;

test de consumo React;

test de consumo Terminal;

test de versión incompatible;

test de campos obligatorios ausentes.

Deberán probarse:

SessionStatus;

MenuState;

ModelCatalog;

CommandResult;

WorkspaceState;

PipelineState;

ClaimState;

ContextEnvelope;

ContextReceipt;

ErrorState;

WelcomeState;

ContradictionRecord;

Capability;

EvidenceRecord.

Una modificación contractual no podrá integrarse si uno de los consumidores deja de validar.

85. EQUIVALENCIA ENTRE TERMINAL Y REACT

Para el mismo estado y la misma acción canónica deberán coincidir:

command_id;

WorkspaceState;

SessionStatus final;

execution_state;

modelo efectivo;

workspace_id;

context_revision;

receipt_ids;

errores semánticos;

selección efectiva;

allowed_actions;

contradiction_ids.

Podrán diferir:

formato;

colores;

densidad;

disposición;

interacción visual.

No podrán diferir:

autoridad;

resultado;

permisos;

evidencia;

certificación;

acción ejecutada.

86. CONTRATO DE DISTRIBUCIÓN

Un artefacto distribuible deberá ser limpio y reproducible.

No deberá incluir:

node_modules;

logs de desarrollo;

state de sesiones reales;

credenciales;

cachés;

bytecode;

pycache;

archivos temporales;

receipts privados;

artefactos locales no declarados.

Deberá incluir:

código necesario;

lockfiles;

manifiesto de distribución;

versión;

hashes;

instrucciones reproducibles;

esquemas contractuales;

migraciones necesarias.

Las dependencias frontend se reinstalarán desde el lockfile durante el build.

87. REPRODUCIBILIDAD DEL BUILD

El build deberá reproducirse desde un entorno limpio.

La prueba deberá:

extraer el paquete;

instalar dependencias declaradas;

generar tipos;

validar contratos;

construir backend;

construir frontend;

ejecutar self-tests;

ejecutar suite mantenida;

ejecutar pruebas de integridad;

crear artefacto;

calcular hashes.

Un paquete que solo funcione con dependencias residuales no será reproducible.

88. LIMPIEZA PROGRESIVA Y COMPATIBILIDAD TEMPORAL

La limpieza deberá ejecutarse mediante sustitución progresiva.

La regla será:

IDENTIFICAR.

PROTEGER CON TESTS.

CREAR REEMPLAZO CANÓNICO.

MIGRAR CONSUMIDORES.

ELIMINAR LA PIEZA ANTIGUA.

88.1. Clasificación

Cada pieza será clasificada como:

canónica;

compatibilidad temporal;

estado local de presentación;

autoridad local falsa;

código muerto.

88.2. Compatibilidad temporal

Una capa temporal deberá:

traducir contratos;

no recalcular autoridad;

no decidir permisos;

no escribir mediante reglas legacy;

emitir deprecación;

registrar consumidores;

tener condición de retirada.

88.3. Deuda de compatibilidad

Cada shim deberá registrar:

compat_id;

ruta;

símbolo antiguo;

consumidores pendientes;

contrato sustituto;

riesgo;

test;

condición de eliminación;

estado.

88.4. Puerta de retirada

Una pieza legacy podrá eliminarse cuando:

no tenga consumidores;

no tenga imports;

no tenga referencias dinámicas conocidas;

el contrato sustituto esté activo;

los tests canónicos pasen;

la mutación sea detectada;

exista rollback.

89. CRITERIOS MÍNIMOS PARA CERRAR ESTA ITERACIÓN

La iteración podrá cerrarse cuando se haya demostrado que:

uno, `framework_root` corresponde a la instalación real;

dos, la sesión está vinculada al `workspace_root` real;

tres, `project_root`, `workspace_root` y `workspace_scope_root` están diferenciados;

cuatro, `workspace_root` corresponde a `project_root/.gabo`;

cinco, existe `.gabo/workspace.json` para un workspace válido;

seis, existe un único `WorkspaceState`;

siete, el vínculo persiste después de reiniciar;

ocho, cambiar de proyecto exige revinculación;

nueve, detectar `.gabo` válida muestra el workspace;

diez, detectar `.gabo` válida no ofrece inicialización;

once, una sesión no vinculada ofrece vinculación;

doce, una `.gabo` inválida ofrece verificación o reparación;

trece, `.bago` legacy no compite con `.gabo` ;

catorce, BAGO conserva objetivo y decisiones;

quince, los mensajes breves se interpretan dentro del contexto;

dieciséis, existe preflight antes de operaciones de proyecto;

diecisiete, no se llama al modelo de proyecto con binding no confirmado;

dieciocho, versión, modelo, provider, sesión y workspace proceden del backend;

diecinueve, el modelo efectivo coincide con el observado;

veinte, existe ContextEnvelope antes de cada llamada relevante;

veintiuno, ContextEnvelope incluye autoridades correctas;

veintidós, existe ContextReceipt después de la llamada;

veintitrés, el receipt se valida antes de usarlo;

veinticuatro, la recuperación respeta workspace_scope_root;

veinticinco, los fragmentos registran procedencia;

veintiséis, BAGO reconoce lo que desconoce;

veintisiete, no inventa rutas, archivos, herramientas o resultados;

veintiocho, el pipeline se controla desde el backend;

veintinueve, los claims se controlan desde el backend;

treinta, Terminal y React consumen DTOs canónicos;

treinta y uno, existe una fuente versionada de contratos;

treinta y dos, backend, React y Terminal validan contratos;

treinta y tres, el menú se genera desde el registro canónico;

treinta y cuatro, React no mantiene estado operacional autoritativo;

treinta y cinco, ModelCatalog tiene forma única;

treinta y seis, SessionStatus tiene forma única;

treinta y siete, los centros muestran estado, colección, selección y acciones;

treinta y ocho, el primer nivel contiene dominios y no verbos repetidos;

treinta y nueve, el registro y el árbol son distintos pero derivados;

cuarenta, Terminal y React consumen el mismo MenuState;

cuarenta y uno, toda acción visible tiene command_id;

cuarenta y dos, la selección visual requiere confirmación;

cuarenta y tres, acción interactiva y comando directo son equivalentes;

cuarenta y cuatro, los submenús requieren necesidad demostrada;

cuarenta y cinco, el árbol ordinario no supera tres niveles sin justificación;

cuarenta y seis, no existe un registro manual paralelo;

cuarenta y siete, los estados críticos usan constructores y validadores;

cuarenta y ocho, done no puede construirse sin receipt;

cuarenta y nueve, certified no puede construirse sin fingerprint;

cincuenta, existe cadena de eventos verificable;

cincuenta y uno, las evidencias conservan procedencia y hash;

cincuenta y dos, las contradicciones se registran;

cincuenta y tres, las contradicciones críticas se contienen;

cincuenta y cuatro, las operaciones críticas utilizan capabilities;

cincuenta y cinco, las capabilities caducan y pueden revocarse;

cincuenta y seis, los efectos críticos se verifican;

cincuenta y siete, los componentes están identificados;

cincuenta y ocho, no existen defaults engañosos;

cincuenta y nueve, las excepciones críticas no se silencian;

sesenta, existe certificación independiente;

sesenta y uno, las certificaciones se invalidan;

sesenta y dos, las pruebas inversas detectan incumplimientos;

sesenta y tres, la suite adversarial detecta falsedades deliberadas;

sesenta y cuatro, la suite mantenida excluye scripts one-off;

sesenta y cinco, el build es reproducible;

sesenta y seis, no existen regresiones critical conocidas.

Estos requisitos deberán estar, como mínimo, `verified_in_integration`.

Las capacidades no certificadas deberán declararse como pendientes.

90. CIERRE DE ESTA ITERACIÓN

La fórmula será:

ITERACIÓN CERRADA = BASE OPERACIONAL FIABLE Y CRITERIOS MÍNIMOS DEMOSTRADOS Y CONTRATOS COMPATIBLES Y MENÚ CANÓNICO COHERENTE Y ESTADOS DIFÍCILES DE FALSIFICAR Y EVIDENCIAS ÍNTEGRAS Y CONTRADICCIONES CONTENIDAS Y PRUEBAS NEGATIVAS Y MUTACIONES DETECTADAS Y BUILD REPRODUCIBLE Y AUSENCIA DE REGRESIONES CRÍTICAS Y PENDIENTES DECLARADOS

91. CIERRE DEL CANON COMPLETO

La fórmula será:

CANON CERRADO = TODOS LOS REQUISITOS OBLIGATORIOS CERTIFICADOS Y RESULTADOS REPRODUCIBLES Y CONTRATOS VERSIONADOS Y RESISTENCIA A LA FALSEDAD DEMOSTRADA Y SIN BLOQUEOS CRÍTICOS Y SIN INCOHERENCIAS DE ESTADO Y SIN REGRESIONES CONOCIDAS

Hasta entonces, los estados permitidos serán:

BASE OPERACIONAL VERIFICADA;

CANON PARCIALMENTE CERTIFICADO;

CANON ABIERTO.

92. ENTREGA OBLIGATORIA DEL PROGRAMADOR

Antes de modificar:

mapa de arquitectura;

rutas verificadas;

autoridades detectadas;

contratos existentes;

productores y consumidores;

registro de comandos;

árboles visibles existentes;

componentes reutilizables;

capabilities existentes;

eventos existentes;

brechas confirmadas;

riesgos;

plan por fases;

claims afectados;

tests que deben fallar inicialmente.

Después de cada fase:

archivos modificados;

claims;

contratos modificados;

tipos regenerados;

comandos registrados;

centros afectados;

capabilities modificadas;

eventos añadidos;

tests añadidos;

tests ejecutados;

resultados;

evidencias;

contradicciones;

fallos;

regresiones;

aspectos no comprobados;

siguiente puerta.

Al finalizar:

diff completo;

comandos;

códigos de salida;

logs;

tests pasados;

tests fallidos;

tests omitidos;

mutaciones;

fixtures contractuales;

ContextEnvelope de muestra;

ContextReceipt de muestra;

WorkspaceState de muestra;

MenuState de muestra;

SessionStatus de muestra;

ContradictionRecord de muestra;

Capability de muestra;

EvidenceRecord de muestra;

cadena de eventos de muestra;

equivalencia de acciones directas e interactivas;

mediciones observadas;

datos calculados;

datos estimados;

procedimiento de reversión;

riesgo residual;

estado de ejecución;

estado de madurez;

estado de certificación;

estado de iteración;

estado del canon.

93. ENTREGA OBLIGATORIA DEL CERTIFICADOR

El certificador entregará:

ESTADO FINAL.

AFIRMACIONES COMPROBADAS.

PRUEBAS EJECUTADAS.

ENTORNO Y HUELLA.

CONTRATOS VALIDADOS.

CENTROS OPERATIVOS VALIDADOS.

EQUIVALENCIA DE ACCIONES.

INTEGRIDAD DE EVENTOS.

CAPABILITIES VALIDADAS.

CONTRADICCIONES ABIERTAS Y RESUELTAS.

EVIDENCIAS.

FALLOS ENCONTRADOS.

MUTACIONES.

REGRESIONES.

ASPECTOS NO COMPROBADOS.

RIESGO RESIDUAL.

CONCLUSIÓN.

Deberá comprobar específicamente:

los seis WorkspaceState;

bienvenidas adaptativas;

opciones del menú por estado;

rechazo de init con `.gabo` válida;

preflight antes del modelo;

equivalencia Terminal y React;

contratos productor-consumidor;

pipeline controlado por backend;

claims controlados por backend;

centros operativos;

selección confirmada;

acciones directas e interactivas;

profundidad del árbol;

receipts validados;

cadena de eventos;

capabilities;

contradicciones;

pruebas inversas;

build limpio y reproducible.

No utilizará expresiones ambiguas.

Indicará exactamente qué está demostrado y qué no.

94. COMPORTAMIENTO FINAL ESPERADO

Cuando BAGO detecte una `.gabo` válida y vinculada, informará:

en qué proyecto está;

qué workspace utiliza;

qué sesión está vinculada;

qué modelo está activo;

qué contexto está confirmado;

qué objetivo mantiene.

No preguntará si el usuario desea crear otro `.gabo`.

Cuando la sesión no esté vinculada, ofrecerá vincularla.

Cuando el manifiesto sea incoherente, ofrecerá verificar o reparar.

Cuando `.gabo` no exista, podrá ofrecer inicialización.

Cuando exista únicamente `.bago` legacy, ofrecerá migración.

Al abrir `/`, BAGO mostrará centros operativos compactos.

Al abrir un centro, mostrará estado, colección, selección y acciones.

Los comandos directos continuarán disponibles.

Cuando exista una contradicción crítica, BAGO la mostrará y bloqueará el dominio afectado.

Terminal y React podrán mostrar interfaces distintas, pero comunicarán el mismo estado real.

Cuando el usuario pregunte si BAGO está terminado, BAGO informará:

estado de la iteración;

estado del canon;

capacidades implementadas;

capacidades verificadas;

capacidades certificadas;

capacidades rechazadas;

capacidades pendientes;

contradicciones;

riesgos;

límites de evidencia.

95. PREGUNTAS QUE BAGO DEBERÁ PODER RESPONDER

BAGO deberá responder con evidencia:

¿Cuál es `framework_root`?

¿Cuál es `project_root`?

¿Cuál es `workspace_root`?

¿Cuál es `workspace_scope_root`?

¿Qué proyecto estoy manejando?

¿Qué workspace está vinculado?

¿Existe `.gabo`?

¿Es válida?

¿La sesión está vinculada?

¿Cuál es WorkspaceState?

¿Qué acciones están permitidas?

¿Cuál es la raíz autorizada?

¿Cuál es el objetivo actual?

¿Qué decisiones siguen vigentes?

¿Qué modelo recibió el mensaje?

¿Qué provider, adapter y runtime se utilizaron?

¿Qué contrato utiliza la UI?

¿Qué archivos y fragmentos se entregaron?

¿Qué dato no conoce?

¿Qué herramientas están autorizadas?

¿Qué acción se ejecutó?

¿Qué resultado se validó?

¿Cuánto contexto puede utilizar?

¿Cuánto contexto queda?

¿Qué factor limita el contexto?

¿Qué pruebas respaldan la cifra?

¿Qué cambio invalidaría la certificación?

¿Por qué aparece o no `/workspace init`?

¿React y Terminal representan el mismo estado?

¿Qué centro operativo está activo?

¿Qué elemento está seleccionado visualmente?

¿Qué elemento está confirmado efectivamente?

¿Qué command_id respalda cada acción visible?

¿Por qué existe un submenú?

¿La acción directa y la interactiva utilizan el mismo servicio?

¿Qué receipt respalda esta afirmación?

¿El receipt está emitido, validado, contradicho o certificado?

¿Qué testigo observó el efecto?

¿Existe una contradicción abierta?

¿Qué capability autorizó la operación?

¿La cadena de eventos conserva su integridad?

96. CLÁUSULA FINAL DE AUTORIDAD

Si una afirmación sobre framework, proyecto, workspace, sesión o estado contradice al backend, prevalecerá el estado canónico validado.

Si el backend no puede demostrarla, BAGO declarará:

no lo sé;

no está disponible;

no está confirmado;

está bloqueado;

está contradicho.

Si un archivo está fuera de workspace_scope_root, no podrá utilizarse como evidencia sin autorización.

Si una pieza vive en `.bago` y es específica del proyecto, está mal ubicada.

Si una pieza vive en `.gabo` y es genérica del framework, está mal ubicada.

Si `.bago` legacy y `.gabo` existen, `.gabo` prevalece.

Si la UI muestra una ruta que no coincide con el backend, la UI está equivocada.

Si una sesión no puede demostrar qué workspace utiliza, no podrá actuar sobre el proyecto.

Si `.gabo` existe y es válida, BAGO no ofrecerá crear otra.

Si `.gabo` existe pero la sesión no está vinculada, ofrecerá vincular, no inicializar.

Si `.gabo` es inválida, deberá reparar, verificar o bloquear.

Si Terminal y React discrepan, prevalecerá el estado canónico validado.

Si un contrato no puede validarse, el consumidor no interpretará el payload como confirmado.

Si una acción no tiene `command_id` canónico, no podrá mostrarse como ejecutable.

Si una selección no está confirmada por el backend, seguirá siendo visual.

Si el árbol visible contradice `allowed_actions`, el árbol estará equivocado.

Si un receipt contradice un efecto observado, el receipt estará contradicho.

Si el backend contradice un testigo fiable, deberá abrirse una contradicción.

La política canónica será:

`.bago` = framework BAGO.

`project_root` = raíz real del proyecto.

`.gabo` = estado operacional del workspace.

`workspace_scope_root` = límite autorizado.

Backend validado = autoridad operacional.

Contratos versionados = autoridad de frontera.

Registro canónico = autoridad de capacidad ejecutable.

MenuState = semántica contextual del menú.

Árbol visible = proyección de MenuState.

Centros operativos = superficies compactas por dominio.

Comandos directos = acceso reproducible a acciones canónicas.

Terminal y React = presentadores.

LocalStorage = preferencias visuales.

Receipts = declaraciones estructuradas que requieren validación.

Evidencias y testigos = comprobación de efectos.

ContradictionRecord = autoridad sobre desacuerdos abiertos.

Cadena de integridad = protección del historial.

97. INSTRUCCIÓN DEFINITIVA

No construyas otro BAGO paralelo.

No construyas otra autoridad dentro de React.

No mantengas dos registros de comandos.

No mantengas un árbol visual independiente.

No mantengas dos contratos para la misma capacidad.

No sustituyas componentes funcionales sin necesidad demostrada.

No inventes rutas, módulos, comandos o contratos.

No confundas framework root, project root, workspace root y workspace scope root.

No confundas detección con vinculación.

No confundas una `.gabo` existente con un proyecto sin inicializar.

No muestres acciones incompatibles con el estado real.

No ofrezcas `/workspace init` cuando ya existe `.gabo`.

No confundas respuesta HTTP correcta con operación validada.

No confundas receipt emitido con efecto verificado.

No confundas operación terminada con capacidad verificada.

No confundas capacidad verificada con capacidad certificada.

No confundas cierre de iteración con cierre del canon.

No conviertas cada verbo del registro en una rama visible.

No apliques una selección visual sin confirmación.

No implementes por separado la acción directa y la interactiva.

No crees submenús sin carga, riesgo o complejidad demostrada.

No permitas que un componente use su propia afirmación como única evidencia.

No ocultes contradicciones.

No silencies excepciones críticas.

No uses defaults que aparenten estado confirmado.

No permitas una capability fuera de scope.

No declares que BAGO aprende, evoluciona, recupera, mide, ejecuta o certifica sin evidencia reproducible.

No declares como observado un valor estimado.

No declares como ejecutada una acción propuesta.

No declares como vigente una certificación invalidada.

No permitas que la interfaz muestre un estado que el backend no pueda demostrar.

La fórmula final será:

BASE OPERACIONAL FIABLE = AUTORIDAD DE RUTAS Y WORKSPACESTATE ÚNICO Y VINCULACIÓN EXPLÍCITA Y PREFLIGHT OBLIGATORIO Y ESTADO REAL Y CONTRATOS VERSIONADOS Y REGISTRO CANÓNICO ÚNICO Y CENTROS OPERATIVOS Y SELECCIÓN CONFIRMADA Y PRESENTADORES NO AUTORITATIVOS Y CONTEXTO TRAZABLE Y RECUPERACIÓN DEMOSTRADA Y EJECUCIÓN VALIDADA Y RECEIPTS VALIDADOS Y EVIDENCIA REPRODUCIBLE Y CADENA DE INTEGRIDAD Y CAPABILITIES LIMITADAS Y CONTRADICCIONES CONTENIDAS Y UI FIEL AL BACKEND Y MENÚ ADAPTADO AL ESTADO Y BUILD REPRODUCIBLE Y AUSENCIA DE AFIRMACIONES FALSAS

BAGO FIABLE = VERDAD DEMOSTRABLE MÁS FALSEDAD DIFÍCIL DE CONSTRUIR MÁS FALSEDAD FÁCIL DE DETECTAR MÁS CONTRADICCIÓN CONTENIDA MÁS EVIDENCIA ÍNTEGRA MÁS RESPONSABILIDAD ATRIBUIBLE

BAGO no será fiable porque afirme la verdad.

Será fiable cuando la arquitectura haga difícil mentir, difícil ocultarlo y sencillo demostrarlo.

98. CAMBIOS EFECTUADOS EN RC4

Los cambios siguientes han sido efectuados en el documento canónico.

No implican por sí mismos que ya estén implementados en el código.

98.1. Cambio de versión

La versión pasa de 1.0-RC3 a 1.0-RC4.

98.2. Resistencia a la falsedad

Se añade prevención, detección, contención y atribución.

98.3. Estados imposibles

Se prohíben representaciones como done sin receipt o certified sin fingerprint.

98.4. Estados epistemológicos

Se distinguen observed, verified, calculated, estimated, inferred, proposed, unknown y contradicted.

98.5. Receipts validados

Se separa receipt emitido de receipt validado, contradicho y certificado.

98.6. Testigos operacionales

Se introducen observadores independientes de efectos.

98.7. Cadena de integridad

Se añaden eventos append-only enlazados mediante hashes.

98.8. ContradictionRecord

Las contradicciones pasan a ser objetos canónicos.

98.9. Motor de coherencia

Se añade validación transversal de invariantes.

98.10. Capabilities

Se introducen permisos limitados, revocables, vinculados a scope y ejecución.

98.11. Confianza cero

Ninguna capa confía automáticamente en otra.

98.12. Caducidad de evidencias

Las evidencias pueden quedar stale o invalidated.

98.13. Atribución por componente

Todo productor de estado canónico debe estar identificado.

98.14. Prueba de inversión

Toda garantía crítica debe demostrar que detecta su incumplimiento.

98.15. Suite de falsedad

Se añaden pruebas específicas para intentar engañar al sistema.

99. PRUEBAS OBLIGATORIAS DE INCOHERENCIA RC3

Cada prueba deberá producir:

test_id;

estado inicial;

mutación;

resultado esperado;

resultado observado;

evidencia;

PASS, FAIL, BLOCKED o INCONCLUSIVE.

INC-001. Framework root contradictorio

Resultado esperado:

prevalece el backend validado;

la copia local queda stale.

INC-002. Workspace root no corresponde a `.gabo`

Resultado esperado:

WorkspaceState invalid;

preflight bloqueado.

INC-003. `.gabo` sin manifiesto

Resultado esperado:

WorkspaceState invalid.

INC-004. Workspace ID contradictorio

Resultado esperado:

blocked;

sin modelo ni herramientas.

INC-005. Scope fuera del proyecto

Resultado esperado:

binding rechazado.

INC-006. Veredictos de binding divergentes

Resultado esperado:

solo se acepta el resolver canónico.

INC-007. Context revision obsoleta

Resultado esperado:

ContextEnvelope rechazado.

INC-008. Envelope y sesión no coinciden

Resultado esperado:

sin llamada al adapter.

INC-009. Receipt y envelope no coinciden

Resultado esperado:

ejecución no validada.

INC-010. Recuperación fuera del alcance

Resultado esperado:

fragmento rechazado.

INC-011. Fragmento de otro workspace

Resultado esperado:

fragmento descartado.

INC-012. Fragmento obsoleto

Resultado esperado:

reindexado requerido.

INC-013. Archivo del framework como evidencia del proyecto

Resultado esperado:

evidencia rechazada.

INC-014. `.gabo` válida muestra init

Resultado esperado:

test de menú fallido.

INC-015. Workspace detectado no vinculado muestra creación

Resultado esperado:

test fallido.

INC-016. `.gabo` inválida crea otra estructura

Resultado esperado:

operación rechazada.

INC-017. Legacy compite con `.gabo`

Resultado esperado:

`.gabo` prevalece.

INC-018. Menú React diferente del Terminal

Resultado esperado:

test de equivalencia fallido.

INC-019. Comando visual inexistente

Resultado esperado:

acción no representada.

INC-020. ModelCatalog incompatible

Resultado esperado:

validación contractual fallida.

INC-021. SessionStatus ambiguo

Resultado esperado:

test de tipos fallido.

INC-022. Contract version incompatible

Resultado esperado:

contract_incompatible.

INC-023. Campo obligatorio ausente

Resultado esperado:

payload rechazado.

INC-024. Pipeline completado antes del backend

Resultado esperado:

transición rechazada.

INC-025. Pipeline done sin receipt

Resultado esperado:

failed o blocked.

INC-026. Claim manual aprobado

Resultado esperado:

solo borrador.

INC-027. Modelo mostrado distinto del efectivo

Resultado esperado:

degraded y contradicción.

INC-028. Provider, adapter y runtime mezclados

Resultado esperado:

test contractual fallido.

INC-029. LocalStorage contradice al backend

Resultado esperado:

se ignora como autoridad.

INC-030. Canal amplía permisos

Resultado esperado:

test de seguridad fallido.

INC-031. Resultado distinto entre superficies

Resultado esperado:

test de equivalencia fallido.

INC-032. HTTP correcto con ejecución fallida

Resultado esperado:

UI muestra failed.

INC-033. Error oculto en data

Resultado esperado:

validación fallida.

INC-034. Certificación emitida por implementador

Resultado esperado:

certification_state pending.

INC-035. Fingerprint modificada

Resultado esperado:

certificación invalidated.

INC-036. Conteo aproximado presentado como exacto

Resultado esperado:

marcado estimated.

INC-037. Contexto aplicado menor que solicitado

Resultado esperado:

medición invalidada.

INC-038. Reserva fija presentada como medida

Resultado esperado:

configured o estimated.

INC-039. Reintento duplica modificación

Resultado esperado:

no se duplica efecto.

INC-040. Cancelación deja done

Resultado esperado:

cancelled o failed, nunca done.

INC-041. One-off recogido como suite mantenida

Resultado esperado:

la configuración lo impide.

INC-042. Ruta absoluta de desarrollador

Resultado esperado:

test de portabilidad fallido.

INC-043. Separadores dependientes del sistema

Resultado esperado:

normalización multiplataforma.

INC-044. Paquete incluye estado privado

Resultado esperado:

distribución rechazada.

INC-045. Paquete incluye node_modules

Resultado esperado:

distribución rechazada.

INC-046. Build depende de residuos locales

Resultado esperado:

prueba limpia fallida.

INC-047. Tipos generados desactualizados

Resultado esperado:

integración continua fallida.

INC-048. Fixture aceptado por backend y rechazado por React

Resultado esperado:

test productor-consumidor fallido.

INC-049. WelcomeState y MenuState se contradicen

Resultado esperado:

test de coherencia fallido.

INC-050. Confirmed sin evidencia suficiente

Resultado esperado:

degraded o blocked.

INC-051. Entrada separada innecesaria por cada verbo

Resultado esperado:

test de carga cognitiva fallido.

INC-052. Centro operativo sin estado actual

Resultado esperado:

test fallido.

INC-053. Selección visual aplicada sin confirmación

Resultado esperado:

pending_confirmation.

INC-054. Acción sin command_id

Resultado esperado:

MenuState rechazado.

INC-055. Acción directa e interactiva divergentes

Resultado esperado:

test de equivalencia fallido.

INC-056. Registro y árbol independientes

Resultado esperado:

test de derivación fallido.

INC-057. Acción fuera de allowed_actions

Resultado esperado:

test de presentación fallido.

INC-058. División sin necesidad demostrada

Resultado esperado:

test de complejidad fallido.

INC-059. Submenú mecánico por verbos

Resultado esperado:

test fallido.

INC-060. Centro activo incompatible

Resultado esperado:

MenuState rechazado.

INC-061. Selección efectiva obsoleta

Resultado esperado:

selección stale.

INC-062. Comando directo usa implementación paralela

Resultado esperado:

test arquitectónico fallido.

INC-063. Árbol visible expone todo el registro

Resultado esperado:

test de proyección fallido.

INC-064. Centro o capacidad inaccesible

Resultado esperado:

test de accesibilidad fallido.

INC-065. Profundidad injustificada

Resultado esperado:

test de profundidad fallido.

INC-066. Acción bloqueada presentada como permitida

Resultado esperado:

MenuState inválido.

INC-067. Estado omitido durante carga

Resultado esperado:

test visual fallido.

INC-068. Selección sin identidad estable

Resultado esperado:

test fallido.

INC-069. Búsqueda altera autoridad

Resultado esperado:

solo cambia presentación.

INC-070. Centro no adapta sus acciones

Resultado esperado:

test contextual fallido.

100. PRUEBAS OBLIGATORIAS DE RESISTENCIA A LA FALSEDAD

FALSE-001. Pipeline done sin receipt

Resultado esperado:

objeto rechazado.

FALSE-002. Claim verified sin evidencia

Resultado esperado:

transición rechazada.

FALSE-003. Certified sin fingerprint

Resultado esperado:

estado inválido.

FALSE-004. Modelo efectivo tomado de la selección visual

Resultado esperado:

no confirmado.

FALSE-005. Herramienta completada solo por afirmación del modelo

Resultado esperado:

ToolState no pasa a completed.

FALSE-006. Receipt declara archivo escrito, pero el hash no cambia

Resultado esperado:

operación contradicted o failed.

FALSE-007. Evento histórico modificado

Resultado esperado:

ruptura de cadena detectada.

FALSE-008. Evento eliminado

Resultado esperado:

secuencia o hash inválido.

FALSE-009. Evidencia de otro workspace

Resultado esperado:

rechazada.

FALSE-010. Excepción convertida en lista vacía

Resultado esperado:

contrato rechazado.

FALSE-011. UI restaura estado operativo desde LocalStorage

Resultado esperado:

stale, nunca confirmed.

FALSE-012. Backend acepta allowed_actions enviados por UI

Resultado esperado:

solicitud rechazada.

FALSE-013. Menú inventa un command_id

Resultado esperado:

MenuState inválido.

FALSE-014. Comando directo evita preflight

Resultado esperado:

test fallido.

FALSE-015. Selección visual cambia modelo efectivo

Resultado esperado:

sin efecto operacional.

FALSE-016. Certificación no se invalida tras cambiar contrato

Resultado esperado:

invalidated.

FALSE-017. Provider declara modelo distinto del runtime observado

Resultado esperado:

ContradictionRecord crítico.

FALSE-018. Receipt contradice al envelope

Resultado esperado:

ejecución no validada.

FALSE-019. Capability fuera de scope

Resultado esperado:

operación bloqueada.

FALSE-020. Componente sin identidad modifica estado

Resultado esperado:

operación rechazada.

FALSE-021. Dos fuentes críticas derivan del mismo valor

Resultado esperado:

doble confirmación insuficiente.

FALSE-022. Evidencia caducada utilizada como vigente

Resultado esperado:

claim unverified o invalidated.

FALSE-023. Test positivo y mutación pasan

Resultado esperado:

garantía rechazada.

FALSE-024. Estimado presentado como observed

Resultado esperado:

contrato de presentación fallido.

FALSE-025. Error crítico ocultado como warning

Resultado esperado:

operación blocked.

FALSE-026. Receipt formalmente válido sin efecto observable

Resultado esperado:

receipt emitted, no validated.

FALSE-027. Receipt generado por el mismo componente sin testigo

Resultado esperado:

confianza insuficiente para efecto crítico.

FALSE-028. Capability reutilizada después de consumirse

Resultado esperado:

operación rechazada.

FALSE-029. Capability revocada

Resultado esperado:

operación rechazada.

FALSE-030. Capability de otro workspace

Resultado esperado:

operación bloqueada.

FALSE-031. Nonce duplicado

Resultado esperado:

posible replay detectado.

FALSE-032. Cadena de eventos sustituida completamente

Resultado esperado:

el hash anclado o segmento anterior no coincide.

FALSE-033. Evento insertado fuera de secuencia

Resultado esperado:

cadena inválida.

FALSE-034. Contradicción crítica ocultada a la UI

Resultado esperado:

test de presentación fallido.

FALSE-035. Estado confirmed con contradicción crítica abierta

Resultado esperado:

estado inválido.

FALSE-036. Archivo modificado fuera del scope

Resultado esperado:

operación rechazada y evidencia registrada.

FALSE-037. Hash posterior calculado antes de escribir

Resultado esperado:

testigo detecta discrepancia.

FALSE-038. Herramienta devuelve éxito sin proceso o efecto

Resultado esperado:

ToolReceipt contradicted.

FALSE-039. Modelo afirma leer un archivo no recuperado

Resultado esperado:

afirmación clasificada como unsupported.

FALSE-040. Certificador utiliza la misma evidencia fabricada por el ejecutor

Resultado esperado:

diversidad de fuentes insuficiente.

FALSE-041. Backend y testigo discrepan

Resultado esperado:

ContradictionRecord.

FALSE-042. ContradictionRecord eliminado sin resolución

Resultado esperado:

integridad o transición inválida.

FALSE-043. Evidencia reescrita conservando evidence_id

Resultado esperado:

hash inconsistente.

FALSE-044. Claim cambia de statement conservando certificación

Resultado esperado:

certificación invalidated.

FALSE-045. Presenter omite el estado epistemológico

Resultado esperado:

contrato de presentación fallido.

FALSE-046. Default operativo introducido durante error

Resultado esperado:

unknown o blocked, nunca confirmed.

FALSE-047. Servicio anónimo emite evento

Resultado esperado:

evento rechazado.

FALSE-048. ErrorState sin component_id

Resultado esperado:

payload inválido para fallo crítico.

FALSE-049. Reintento con nueva idempotency_key duplica efecto

Resultado esperado:

test de deduplicación o advertencia de riesgo.

FALSE-050. Pipeline reconstruido desde eventos incompletos

Resultado esperado:

estado degraded, nunca done.

101. CRITERIO DE APROBACIÓN DE LAS PRUEBAS

La suite se considerará válida cuando:

cada contradicción provoque el fallo esperado;

cada falsedad deliberada sea detectada;

ninguna mutación crítica pase silenciosamente;

las pruebas identifiquen el componente responsable;

se produzcan evidencias reproducibles;

los resultados sean iguales desde entorno limpio;

React y Terminal consuman las mismas fixtures;

los fallos contractuales bloqueen estados confirmados;

las mutaciones se reviertan después de cada ejecución;

las acciones directas e interactivas sean equivalentes;

el árbol visible se derive del registro;

las selecciones visuales no alteren estado sin confirmación;

la complejidad del árbol esté justificada;

la cadena de eventos detecte alteraciones;

las capabilities impidan efectos fuera de scope;

los receipts falsos no sean validados;

las contradicciones críticas se contengan.

Una suite verde no será suficiente si continúa pasando después de introducir una incoherencia o falsedad que debería detectar.

La prueba definitiva será su capacidad para rechazar deliberadamente una implementación incorrecta.